

$\mathcal{P}$ Torch: Narrowing the Gap Between Projection and Gradient–Based Learning

Jeff Cyuzuzo Jambé

Thesis submitted for the degree of
Master of Science in de
ingenieurswetenschappen: artificiële
intelligentie

Supervisor

Prof. dr. ir. Panos Patrinos

Assessor

Prof. dr. ir. Raf Vandebril
Prof. dr. ir. Matthew Blaschko

Assistant-supervisor

Ir. Jan Quan

© 2026 KU Leuven – Faculty of Engineering Science

Published by Jeff Cyuzuzo Jambé,

Faculty of Engineering Science, Kasteelpark Arenberg 1 bus 2200, B-3001 Leuven

All rights reserved. No part of the publication may be reproduced in any form by print, photoprint, microfilm, electronic or any other means without written permission from the publisher. This publication contains the study work of a student in the context of the academic training and assessment. After this assessment no correction of the study work took place.

Contents

Preface	iii
Abstract	iv
Samenvatting	v
List of Figures and Tables	vi
1 Introduction	1
1.1 Related work	3
<i>Target propagation.</i> 3, <i>ADMM-based deep learning.</i> 3,	
<i>Gradient-free optimization frameworks.</i> 3	
2 Neural networks as a feasibility problem	4
2.1 The Intuition	4
2.2 Mathematical description	4
2.3 Illustrative example: XOR classifier as a feasibility problem	6
<i>Problem setup</i> 6, <i>Network architecture</i> 6, <i>Visualization of the</i>	
<i>optimization</i> 7	
3 Scalable Projection-Based Training	11
3.1 Scaling the Linear Layer Projection	12
<i>Fused consensus and bilinear constraint</i> 12, <i>Projection in a different</i>	
<i>geometry</i> 13, <i>Choosing the appropriate geometry</i> 16	
3.2 From Cyclic Projections to a Practical Optimizer	16
<i>Connection to incremental gradient descent and non-linear relaxation</i>	
16, <i>Proximal Projections at the Loss node</i> 18, <i>Extension to</i>	
<i>Mini-Batches and the Full Algorithm</i> 19	
3.3 Revisiting the XOR Example: Practical Execution in $\mathcal{P}$ Torch	20
<i>Forward and Backward Semantics</i> 20, <i>Loss Projection and Target</i>	
<i>Initialization</i> 20, <i>Output-Layer Update</i> 21, <i>Activation function</i>	
<i>Update</i> 22	
3.4 Summary of the $\mathcal{P}$ Torch Algorithm	22

4	Vanishing Target Signals in Deep Networks	23
4.1	Proof of vanishing Targets	23
4.2	Experimental Verification of Local Non-Expansiveness for the Linear Layer	25
5	Experiments	26
5.1	Shallow MLP: MNIST and CIFAR-10	26
5.2	Vanishing target signal and depth	27
5.3	Convolutional networks: small CNN on CIFAR-10	29
5.4	Training a ViT	29
6	Beyond $\mathcal{P}$ Torch	31
A	Advanced Architectural Evaluations	33
A.1	Hybrid architecture	33
A.2	Hybrid optimization	34
A.3	Non-differentiable activation functions	34
B	Practical Considerations for Projection-Based Training	36
B.1	Architecture and Initialization	36
B.2	Optimizer choice	37
	<i>Projection Mechanics</i>	37
B.3	Loss Formulation and Advanced Relaxations	39
C	Details Experiments	40
D	Useful Projections	41
D.1	Linear layers	41
D.2	Functional layers	42
D.3	Activation functions	43
	Bibliography	45
	Code of Conduct	49



Preface

Before starting to talk about projections and weird optimizers, I would like to thank Jan Quan for all of his support and ideas throughout this project, constantly supporting my weird ideas, hearing me out, providing me with incredibly valuable feedback, and formalizing the mathematics. Even more so while writing the associated paper, I am incredibly thankful for the support and working with me through the night to get it over the line; I couldn't have imagined a better assistant-supervisor. I would also like to thank Prof. Panos Patrinos for giving me the opportunity and supporting me in writing a NeurIPS paper, helping me understand how to write an academic paper. Naturally, I would also like to thank my family and friends for providing me with the mental support to not go crazy while writing this.

Jeff Cyuzuzo Jambé



Abstract

Modern deep learning relies almost exclusively on gradient-based optimization. Projection-based optimization, which recasts training as a feasibility problem over architectural, data, and consensus constraints, offers a gradient-free alternative, but the existing framework suffer from high computational and memory costs, poor scaling with depth, and limited integration with modern deep learning infrastructure. We introduce $\mathcal{P}$ Torch, a projection-based framework built directly on PyTorch’s `autograd` engine by reinterpreting each node’s backward pass as a local projection. We establish a connection between cyclic projections and incremental gradient descent, which lets standard first-order optimizers such as Adam and Muon act as nonlinear relaxation schemes, and we reduce peak memory through a fused linear-layer projection that eliminates the per-sample parameter duplication of prior methods. Together with a proximal loss formulation, these make $\mathcal{P}$ Torch competitive with gradient-based baselines on shallow MLPs for MNIST and CIFAR-10. Finally, we prove a vanishing-target theorem showing that the layer-wise learning signal decays with depth, providing a projection-based analogue of the vanishing-gradient problem and clarifying a central barrier to scaling these methods.



Samenvatting

Moderne deep learning steunt vrijwel uitsluitend op gradiëntgebaseerde optimalisatie. Projectiegebaseerde optimalisatie, die het trainen herformuleert als een haalbaarheidsprobleem over architecturale, data- en consensusbeperkingen, biedt een gradiëntvrij alternatief, maar de bestaande frameworks kampen met hoge reken- en geheugenkosten, slechte schaalbaarheid met de diepte, en beperkte integratie met moderne deep-learning-infrastructuur. Wij introduceren $\mathcal{P}$ Torch, een projectiegebaseerd framework dat rechtstreeks op de **autograd**-engine van PyTorch is gebouwd door de backward pass van elke node te herinterpreteren als een lokale projectie. We leggen een verband tussen cyclische projecties en incrementele gradient descent, waardoor standaard eerste-orde-optimizers zoals Adam en Muon kunnen fungeren als niet-lineaire relaxatieschema's, en we verlagen het piekgeheugen door middel van een gefuseerde projecties die de per-sample parameterduplicatie van eerdere methoden elimineert. Dit maakt $\mathcal{P}$ Torch competitief met gradiëntgebaseerde methoden op ondiepe MLP's voor MNIST en CIFAR-10. Ten slotte bewijzen we een verdwijnend-target-stelling die aantoont dat het laaggewijze leersignaal afneemt met de diepte, wat een projectiegebaseerd analoog van het verdwijnende-gradiëntprobleem oplevert en een centrale barrière voor het opschalen van deze methoden verheldert.

List of Figures and Tables

List of Figures

2.1	The XOR dataset in $\mathbb{R}^2$ is not linearly separable.	6
2.2	The neural network used in the XOR example.	7
2.3	Solution sets for the datapoint with $x[0] = 1$, $x[1] = 1$ and $y = 0$	8
2.4	Solution sets for the datapoint with $x[0] = 0$, $x[1] = 0$ and $y = 0$	9
2.5	Solution sets where (x_0, x_1) is either $(0, 1)$ or $(1, 0)$, and $y = 1$	9
2.6	Combined parameter constraints and output projections for the XOR dataset. The left panel shows the intersection of all constraints on the network parameters, yielding the green feasible region where the network perfectly interpolates the data. The right panel illustrates the corresponding output updates that shift the initial predictions into the accepted regions in the output space.	10
3.1	Graphical interpretation of ΔD_k	14
3.2	Initialization of the backward pass: the network output $h^{(3)}$ is coupled to the loss constraint via the proximal operator of Section 3.2.2.	21
3.3	Update of the output linear layer: projection onto the architectural constraint A_3 couples the output target $h^{(3)}$ to the deepest hidden state $h^{(2)}$ and the weights $\theta^{(3)}$	21
3.4	Projection through the activation constraint A_2 , which carries no parameters.	22
4.1	Progression of the non-expansiveness ratio $r^{(i)}$ across varying network depths L	25
5.1	MLP validation accuracy vs. optimization step for CIFAR-10 (left) and MNIST (right). Shaded bands indicate ± 1 standard deviation.	26
5.2	MLP validation accuracy vs. wall-clock time for CIFAR-10 (left) and MNIST (right).	27
5.3	Target signal magnitude δ_k^n at step 1 for varying depths (left) Final training MSE vs. depth (right) on the identity-recovery task.	27

5.4	Validation accuracy vs. optimization steps for a 4-layer MLP on MNIST, comparing runs with and without orthogonalization applied to hidden state update. The performance gap relative to Torch widens with depth, consistent with the depth-barrier prediction of Chapter 4.	28
5.5	Applying orthogonalization on the hidden-state residuals stabilises δ_k^n across depths.	28
5.6	Validation accuracy vs. wall-clock time on CIFAR-10 with the CNN architecture.	29
5.7	Architecture of the small ViT used for the CIFAR-10 experiment.	30
5.8	Train and validation accuracy per step for the ViT on CIFAR-10.	30
A.1	Architecture of the CNN with a Torch based feature extractor and $\mathcal{P}$ Torch head.	33
A.2	Training dynamics of the hybrid CNN architecture. The integration of $\mathcal{P}$ Torch projection modules with standard PyTorch gradient-based layers demonstrates that hybrid architectures can be trained effectively.	33
A.3	Training dynamics on CIFAR-10 (left) and MNIST (right) under hybrid optimization.	34
A.4	Validation accuracy versus optimization step for networks equipped with continuous and discontinuous activation functions evaluated on MNIST.	35
B.1	Impact of initialization strategies on the network convergence across varying depths. Training accuracy is shown over 200 steps for networks of depth 2, 4, and 8, comparing three initialization methods: standard normal distribution (std=0.01), He normal, and scaled orthogonal. While all evaluated methods achieve convergence in shallow networks (Depth = 2), the normal distribution initialization degrades significantly as depth increases, failing to converge at Depth = 8.	36
B.2	Comparison of different optimizers on the MNIST benchmark.	37
B.3	Effect of projection norm on validation accuracy over wall-clock time.	37
B.4	Peak memory versus input dimension for a linear-layer. PJAX scales substantially worse than $\mathcal{P}$ Torch and Torch. For small input dimensions, $\mathcal{P}$ Torch uses roughly $10\times$ more memory than Torch, but this gap narrows as the input dimension increases.	38
B.6	Error convergence and validation accuracy as a function of the number of backward passes on the MNIST benchmark.	39
B.7	Replacing hard target-assignment with a proximal-loss step accelerates convergence.	39
B.8	First-order relaxation applied to the hidden states on the XOR benchmark.	39

List of Tables

3.1	Points that maximally change ΔD_k given ϵ	14
-----	--	----

4.1	Non-expansiveness ratio $r^{(i)}$ pooled across layers, seeds, and recorded steps, for depths $L \in \{2, 4, 8, 16, 32\}$ (three seeds).	25
C.1	Default experimental protocol. Per-experiment overrides (maximum step budget, early-stopping patience, network width/depth, loss function, projection norm, and optimizer) are stated in the corresponding paragraphs and reflect the released YAML configs.	40

Introduction

Modern deep learning relies almost exclusively on gradient-based optimization, with gradients computed end-to-end via backpropagation [35]. However, this paradigm imposes two structural assumptions: every component of the computational graph must admit a gradient (or a tractable subgradient), and parameter updates must propagate through the chain rule. Beyond these technical constraints lies a more systemic concern: the overwhelming focus on gradient-based training risks a form of paradigm lock-in, whereby the community may overlook alternative learning frameworks that given comparable research investment could ultimately prove competitive or even superior. Many alternative paradigms have been proposed in the literature, ranging from classical proposals such as target propagation [28], the method of auxiliary coordinates [14] and feedback alignment [30], to more recent schemes like equilibrium propagation [36], forward-forward training [24] and neuroevolution [37]. Nevertheless, none of these paradigms have achieved widespread adoption, typically bottlenecked by their heuristic approximations and inability to computationally scale on modern hardware.

A promising, mathematically grounded alternative that has recently gained attention is to recast learning as a feasibility problem [18, 9, 19]. Instead of minimizing a global loss function $\mathcal{L}$, this approach searches for some point $\tilde{\theta}$ in the intersection of multiple constraint sets. These sets are chosen in a way so that from a feasible point $\tilde{\theta}$, one can derive the parameters θ of the neural network such that the training data is perfectly interpolated. Conceptually, this paradigm shift takes the form

$$\underset{\theta \in \Theta}{\text{minimize}} \mathcal{L}(f(x; \theta), y) \implies \text{find } \tilde{\theta} \in \bigcap_{i=1}^m C_i.$$

A rich literature exists to tackle such problems, where projection-based methods are the most well-known. Examples of such methods include, but are not limited to, cyclic projections [20], Bregman projections [13], Dykstra’s algorithm [16], and averaged projections [3]. In the two-set case, Douglas–Rachford splitting [15, 17] has been studied extensively and applied across a wide range of problems [7, 31]. Convergence theory for these projection-based methods has been primarily developed in the convex setting [4], and there have been recent advances in the nonconvex

setting as well [39, 2, 29].

While these projection-based algorithms are promising, they are currently not competitive with gradient-based methods even for shallow networks. Moreover they come at the cost of substantial memory overhead, which prohibits scaling at the hardware level. Further, the frameworks in [18, 9] were developed largely in isolation from modern deep learning toolchains and therefore cannot benefit from the many software optimizations that underpin state-of-the-art libraries such as Torch [33]. In particular, while PJAX [9] is built on top of JAX [12], it utilizes only the high-performance tensor computation library while all neural network training logic has been completely redefined within PJAX itself. These limitations naturally lead us to our main research question.

Can we design a projection-based training framework native to modern deep learning toolchains that eliminates severe memory bottlenecks, avoids computational overhead, and achieves performance competitive with gradient-based methods?

CONTRIBUTIONS We address these challenges by introducing $\mathcal{P}$ Torch. Concretely, our contributions are as follows.

1. **A PyTorch-native projection framework.** We establish a conceptual link between cyclic projections, existing projection-based learning algorithms [18, 9], and computational graph traversal via PyTorch’s `autograd`. Leveraging this connection, we develop a highly extensible, PyTorch-native projection framework that inherits PyTorch’s software optimizations and enables training hybrid networks that benefit from the properties of both paradigms.
2. **Eliminating memory overhead and improving performance.** To mitigate memory constraints, we introduce a fused projection strategy that avoids parameter duplications of prior methods [18, 9]. Furthermore, we demonstrate that modern first-order optimizers, such as Adam [27] and Muon [26], can be utilized as nonlinear relaxation schemes for cyclic projections. This insight yields substantial empirical gains, enabling our framework to match or exceed the performance of gradient-based baselines on shallow networks.
3. **Vanishing targets.** We prove that the base cyclic projection algorithm suffers from a problem analogous to vanishing gradients, thereby limiting its algorithmic scalability to deeper networks. Successfully scaling these models presents a clear avenue of future work: discovering the projection-based equivalents of innovative heuristics like He initialization [22], batch normalization [25] and architectural modifications like skip connections [23].

1.1 Related work

Besides projection-based learning, a large body of work has explored alternative gradient-free paradigms. We briefly discuss the most prominent ones here.

1.1.1 Target propagation.

Target propagation is a gradient-free learning paradigm where layer-wise targets are propagated backward through the network instead of gradients [28]. While these methods attempt to generate these targets by approximating the inverse of the forward map, our projection-based approach instead treats layer-wise updates as local feasibility problems. Instead of relying on approximate inverses, $\mathcal{P}$ Torch computes targets by mathematically projecting the current state onto the layer’s constraint set to find the smallest adjustment required to satisfy the network’s architectural constraints. Despite these different mechanisms for generating local targets, both approaches share a fundamental limitation: the attenuation of the learning signal with depth.

1.1.2 ADMM-based deep learning.

The alternating direction method of multipliers (ADMM) [32] offers a prominent alternative to the projection-based training of neural networks. Similar to projection-based methods, ADMM decouples the network into per-layer subproblems by introducing auxiliary variables and enforcing consistency constraints via a Lagrangian penalty. The dlADMM [40] algorithm updates parameters in a backward-then-forward sweep to propagate information more efficiently. Crucially, this approach reduces the subproblem complexity from $O(n^3)$ to $O(n^2)$ using iterative quadratic approximations and backtracking, while also establishing rigorous global convergence guarantees under mild conditions on the loss and activation functions. Ultimately, the layer-wise decomposition in dlADMM conceptually aligns with $\mathcal{P}$ Torch’s target propagation: both methods bypass end-to-end differentiation by decomposing the global objective into isolated, layer-wise optimization steps propagated through the computational graph.

1.1.3 Gradient-free optimization frameworks.

A variety of other gradient-free optimization methods have been proposed in the literature, including the method of auxiliary coordinates [14]. By unfolding the network into layers, this approach replaces a complex nested optimization problem with local optimization problems at the level of individual layers. Distinct paradigms such as neuroevolution have also attracted significant attention and even inspired dedicated frameworks [38]. Another notable alternative is the more recent forward-forward algorithm [24]. However, a common thread across these approaches is that none have yet achieved widespread adoption in mainstream deep learning or been able to train modern deep networks as efficiently as gradient-based methods.

Neural networks as a feasibility problem

2.1 The Intuition

Conceptually, a neural network can be viewed as a sequence of interlocking constraints. Each layer imposes a precise mathematical relationship: its output must equal a transformation of its inputs. The overall goal of training is for the network to interpolate or approximate a target dataset. By fixing the network inputs to the data and the outputs to the ground-truth labels, we impose global boundary conditions that generally violate the network's internal constraints. Training therefore amounts to iteratively resolving these local inconsistencies. Rather than computing global gradients that must be propagated through the chain rule, our approach solves a local optimization problem at each layer:

What is the minimal adjustment to the local weights, inputs, and outputs needed to exactly re-satisfy the layer constraint?

By repeatedly applying these local projections from the output back to the input, a learning signal is propagated naturally through the network without requiring gradients.

2.2 Mathematical description

We first consider the deterministic case, where we have access to the full dataset $\{(x_1, y_1), \dots, (x_B, y_B)\}$ with x_b and y_b denoting the input and output of the b th sample respectively. Suppose we have a neural network with parameters $\theta^{(1)}, \dots, \theta^{(N)}$. For ease of presentation we assume the network has a chain structure, although the extension to more complex topologies is straightforward. In particular, the forward pass for a single sample (x_b, y_b) can be written as

$$\begin{aligned} h_b^{(0)} &= x_b \\ h_b^{(i+1)} &= f_{i+1}(h_b^{(i)}, \theta^{(i+1)}), \quad i = 0, \dots, N-1 \end{aligned}$$

for some functions $f_1, \dots, f_N$. The output of the network is given by $h^{(N)}$.

Example 2.1. Let $\theta^{(i)}$ be a matrix, Σ an activation function and $h_b^{(i-1)}$ a vector. Then $f_i(h_b^{(i-1)}, \theta^{(i)}) = \theta^{(i)} h_b^{(i-1)}$ followed by $f_{i+1}(h_b^{(i)}, \theta^{(i+1)}) = \Sigma(h_b^{(i)})$ is the standard

building block of a multilayer perceptron. Note that $\theta^{(i+1)}$ is an “empty” variable in this case.

We now proceed to model neural network training as a feasibility problem in a high-dimensional space. Following the framework of [18, 9], we associate auxiliary variables $\Omega_b = \{h_b^{(0)}, \dots, h_b^{(N)}, \theta_b^{(1)}, \dots, \theta_b^{(N)}\}$ with each datapoint (x_b, y_b) . Such variables are also introduced in [14] where a penalty-based approach is used instead. The feasibility problem then consists of finding an $\omega = (\Omega_1, \dots, \Omega_B)$ in the intersection of some well-chosen constraint sets

$$\text{find } \omega \in \left(\bigcap_{b=1}^B \bigcap_{i=1}^N A_{bi} \right) \cap (B_{\text{in}} \cap B_{\text{out}}) \cap \left(\bigcap_{i=1}^N C_i \right). \quad (\text{P})$$

For neural network training, there are three types of constraints:

1. **Architectural constraints:** Every layer must satisfy its corresponding input-output transformation. In particular, $\omega \in A_{bi}$ ensures that $h_b^{(i)} = f_i(h_b^{(i-1)}, \theta_b^{(i)})$ for the b th datapoint and the i th layer.
2. **Batch (data) constraints:** The inputs and outputs of the network are fixed to match the current training batch, i.e., $\omega \in B_{\text{in}} \cap B_{\text{out}}$ means that $h_b^{(0)} = x_b$ and $h_b^{(N)} = y_b$ for each $b = 1, \dots, B$.
3. **Consensus constraints:** The duplicated parameters must have a single consistent value. Concretely, $\omega \in C_i$ yields $\theta_1^{(i)} = \dots = \theta_B^{(i)}$ for the i th layer.

If the problem is feasible, then an equivalent minimization problem is

$$\underset{\omega \in \Omega}{\text{minimize}} F(\omega)$$

where

$$F(\omega) := \frac{1}{2} \left\{ \sum_{b=1}^B \sum_{i=1}^N \text{dist}^2(A_{bi}, \omega) + \text{dist}^2(B_{\text{in}}, \omega) + \text{dist}^2(B_{\text{out}}, \omega) + \sum_{i=1}^N \text{dist}^2(C_i, \omega) \right\}.$$

Here, $\text{dist}^2(C, \omega) = \min_{\bar{\omega} \in C} \|\omega - \bar{\omega}\|^2$ denotes the squared distance from ω to a closed set C . Indeed, because the squared distance to any set is nonnegative, $F(\omega^*) = 0$ if and only if ω^* perfectly satisfies all constraints simultaneously, meaning it lies in the intersection of all defined sets.

To summarize, if we find an ω solving (P), then the parameters $\theta^{(1)}, \dots, \theta^{(N)}$ (which are equal over all datapoints due to the consensus constraint) allow the neural network to perfectly interpolate the training data.

When dealing with feasibility problems, we rely on projections instead of gradients. Specifically, the Euclidean projection of a point ω onto a set C is defined as $\Pi_C(\omega) =$

$\arg \min_{\bar{\omega} \in C} \|\omega - \bar{\omega}\|_2$. It is important to note that if C is non-convex, this minimizer is not necessarily unique, making $\Pi_C(\omega)$ a set of points rather than a single value. This projection operator is connected to the gradient of the squared distance function $\text{dist}^2(C, \omega)$, a relationship we will exploit later to motivate a nonlinearly relaxed variant of our proposed method.

Example 2.2. For the consensus set $C = \{(x_1, \dots, x_B) : x_1 = \dots = x_B\}$, we have that $\Pi_C((y_1, \dots, y_B)) = (\tilde{y}, \dots, \tilde{y})$ where $\tilde{y}$ is the average $\frac{1}{B} \sum_{i=1}^B y_i$. Other well-known projections that are useful for neural networks can be found in Appendix D.

2.3 Illustrative example: XOR classifier as a feasibility problem

To clarify the notation from the preceding section and get a concrete understanding of how the feasibility-based approach works, we study an XOR example.

2.3.1 Problem setup

Inspired by the boolean classification example in [18], we use the XOR classification task because it is simple to visualize yet not linearly separable, which makes it a convenient toy example for neural network optimization. We consider the dataset $\{(x_b, y_b)\}_{b=1}^B$ with $B = 4$, binary inputs $x_b = (x_b[0], x_b[1]) \in \{0, 1\}^2$ and labels

$$y_b = x_b[0] \oplus x_b[1],$$

where $\oplus$ denotes XOR. The four points form the classic nonlinearly separable pattern shown in Figure 2.1.

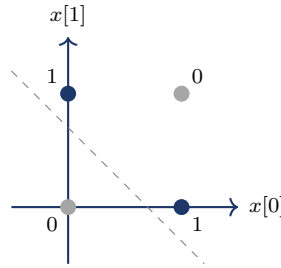


FIGURE 2.1: The XOR dataset in $\mathbb{R}^2$ is not linearly separable.

2.3.2 Network architecture

We employ a multilayer perceptron with a single hidden layer to solve the XOR task. The forward mapping for an input x_b is defined:

$$h^{(1)} = \theta^{(1)} \begin{bmatrix} x_b \\ 1 \end{bmatrix}, \quad h^{(2)} = \Sigma(h^{(1)}) = \begin{cases} 1 & \text{if } h^{(1)} \geq 0 \\ 0 & \text{if } h^{(1)} < 0 \end{cases}, \quad h^{(3)} = \theta^{(3)} \begin{bmatrix} h^{(2)} \\ 1 \end{bmatrix}$$

where $\theta^{(1)} \in \mathbb{R}^{2 \times 3}$ is the hidden layer weight matrix, $\Sigma(\cdot)$ is the step activation function, and $\theta^{(3)} \in \mathbb{R}^{1 \times 3}$ is the output layer weight matrix. Instead of adding bias parameters, we appended a 1 to the input vectors and hidden activations. This network structure is visualized in Figure 2.2.

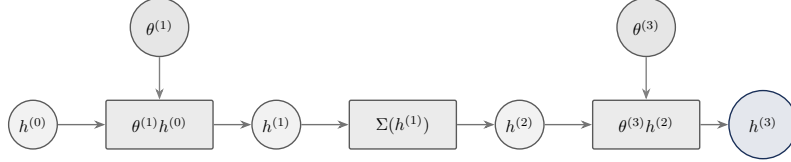


FIGURE 2.2: The neural network used in the XOR example.

To obtain a problem that can be visualized in 2D, we freeze all but two parameters. In particular, the first layer $\theta^{(1)}$ has been fixed to its optimal value and we freeze the middle weight of the output layer at its initial value $\theta^{(3)}[1] = -2$. The two other output layer parameters, $\theta^{(3)}[0]$ and $\theta^{(3)}[2]$ are the only ones being optimized. $\theta^{(2)}$ corresponds to the activation-function layer, which is empty since it has no trainable weights, cf. Example 2.1. The initial parameter values before training are:

$$\theta_{\text{optimal}}^{(1)} = \begin{bmatrix} 1 & 1 & -1 \\ 1 & 1 & -2 \end{bmatrix}, \quad \theta_{\text{init}}^{(3)} = \begin{bmatrix} 1 & -2 & -1 \end{bmatrix}.$$

For the output batch constraint, instead of requiring the output to equal the data exactly, we require the scores for positive instances ($y = 1$) to exceed 0 by at least 1, and for negative instances ($y = 0$) to be strictly less than 0.

2.3.3 Visualization of the optimization

We now optimize the parameters and visualize their associated feasible regions.

DATAPOINT $b = 1$ WITH $x_1[0] = 1$ $x_1[1] = 1$

$$h_1^{(1)} = \theta_{\text{optimal}}^{(1)} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad h_1^{(2)} = \Sigma(h_1^{(1)}) = \begin{bmatrix} 1 \\ 1 \end{bmatrix},$$

1. **Batch constraint:** Since $x_1[0] = x_1[1]$, the XOR label equals $y_1 = 0$. The output constraint therefore requires the output to lie on the negative side of the decision boundary:

$$h_1^{(3)} \leq 0.$$

2. **Architectural constraint:** The output layer must be a matrix multiplication:

$$h_1^{(3)} = \theta_1^{(3)} \begin{bmatrix} h_1^{(2)} \\ 1 \end{bmatrix} = \theta_1^{(3)}[0] - 2 + \theta_1^{(3)}[2],$$

where the constant -2 is the contribution of the frozen middle weight.

Intersecting the batch and architectural constraints gives the feasible set for the parameters associated to datapoint 1:

$$(\theta_1^{(3)}, h_1^{(3)}) \in A_{13} \cap B_{\text{out}} \quad A_{13} \cap B_{\text{out}} := \{(\theta, h) : h = \theta[0] - 2 + \theta[2], h \leq 0\}.$$

The blue regions in Figure 2.3 mark the feasible regions. Any weight combination within this region results in an output less than 0 when $h_1^{(2)}$ is provided as input, thereby satisfying the network's output constraints.

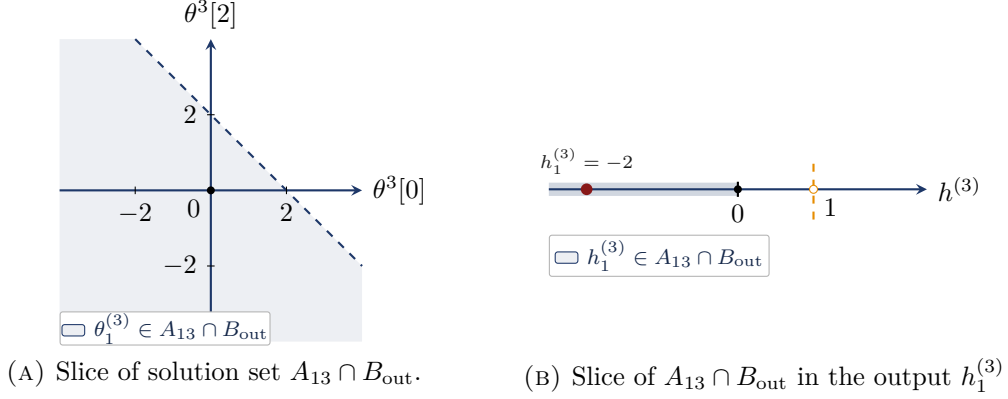


FIGURE 2.3: Solution sets for the datapoint with $x[0] = 1$, $x[1] = 1$ and $y = 0$.

Next, we determine the feasible regions for the remaining datapoints. The intersection of all these areas defines a region where any selected weight allows the model to interpolate the data perfectly.

DATAPOINT $b = 2$ WITH $x_2[0] = 0$, $x_2[1] = 0$.

$$h_2^{(1)} = \theta_{\text{optimal}}^{(1)} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ -2 \end{bmatrix}, \quad h_2^{(2)} = \Sigma(h_2^{(1)}) = \begin{bmatrix} 0 \\ 0 \end{bmatrix},$$

The batch constraint is $h_2^{(3)} \leq 0$ and the architectural constraint $h_2^{(3)} = \theta_2^{(3)}[2]$

$$(\theta_2^{(3)}, h_2^{(3)}) \in A_{23} \cap B_{\text{out}} \quad A_{23} \cap B_{\text{out}} := \{(\theta, h) : h = \theta[2], h \leq 0\}.$$

These regions are highlighted in Figure 2.4.

Notice that for $b = 1, 2$, the initial outputs $h_1^{(3)}$ and $h_2^{(3)}$ are -2 and -1 . These values, highlighted with a red dot in Figure 2.3 and Figure 2.4, are already feasible; thus, there needs to be no change to the parameters for these two datapoints.

2.3. Illustrative example: XOR classifier as a feasibility problem

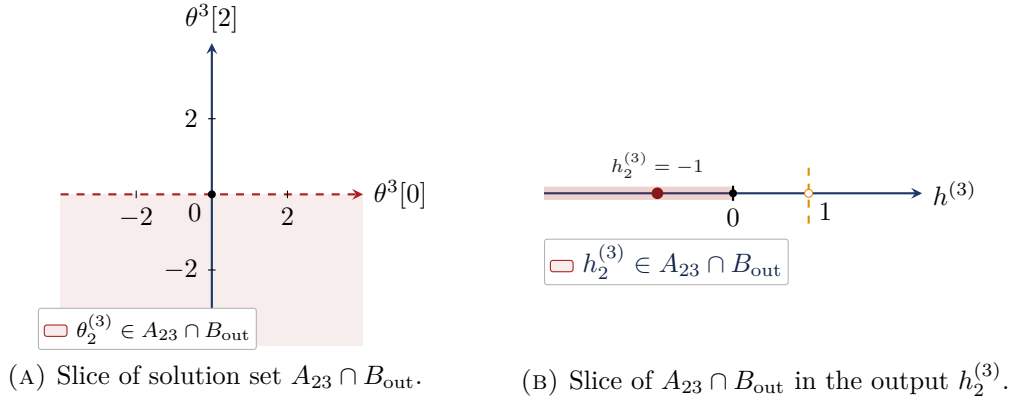


FIGURE 2.4: Solution sets for the datapoint with $x[0] = 0$, $x[1] = 0$ and $y = 0$.

DATAPPOINTS $b = 3, 4$ WITH $x_3 = (0, 1)$, $x_4 = (1, 0)$.

$$h_{(3,4)}^{(1)} = \theta_{\text{optimal}}^{(1)} h_{(3,4)}^{(0)} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \quad h_{(3,4)}^{(2)} = \Sigma(h_{(3,4)}^{(1)}) = \begin{bmatrix} 1 \\ 0 \end{bmatrix},$$

Both datapoints map to the same hidden activation and the same label $y = 1$, so they impose an identical constraint, $h^{(3)} \geq 1$ with $h^{(3)} = \theta^{(3)}[0] + \theta^{(3)}[2]$:

$$(\theta_{(3,4)}^{(3)}, h_{(3,4)}^{(3)}) \in A_{33} \cap B_{out} = A_{43} \cap B_{out}, \quad A_{33} \cap B_{out} := \{(\theta, h) : h = \theta[0] + \theta[2], h \geq 1\}$$

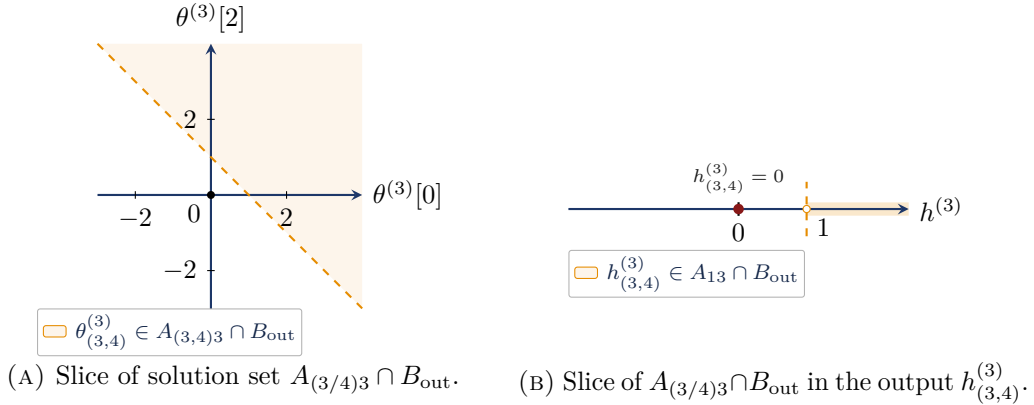


FIGURE 2.5: Solution sets where (x_0, x_1) is either $(0, 1)$ or $(1, 0)$, and $y = 1$.

The initial output $h_{(3,4)}^{(3)} = 0$ violates $h_{(3,4)}^{(3)} \geq 1$ and must be updated to have a network which interpolates the data.

CONSENSUS BETWEEN SAMPLES. Each A_{b3} constrains its own per-sample weight copy $\theta_b^{(3)}$. The consensus constraint C_3 ties these copies together thus a globally feasible solution must be in the intersection of the previously described architectural and batch constraints and the consensus constraint over all samples of the data. The overall problem (P) becomes

$$(\theta_1^{(3)}[0], \dots, \theta_4^{(3)}[0], \theta_1^{(3)}[2], \dots, \theta_4^{(3)}[2], h_1^{(3)}, \dots, h_4^{(3)}) \in B_{out} \cap C_3 \cap \bigcap_{b=1}^4 A_{b3}.$$

The figures below overlays the constraints from the previous figures. Collecting the four slices associated to each datapoint, the combined feasible region is highlighted in green. Any point in the highlighted region satisfies all four datapoint constraints simultaneously and therefore corresponds to weights that perfectly interpolate the training data.

A possible weight update could be moving $\theta^{(3)}[0]$ from 1 to 2, as shown in Figure 2.6. This causes the outputs of data points $b = 1, 2$ to move without exceeding 0, and the outputs of $b = 3, 4$ to move so that they are larger than 1, resulting in a network that perfectly interpolates the data.

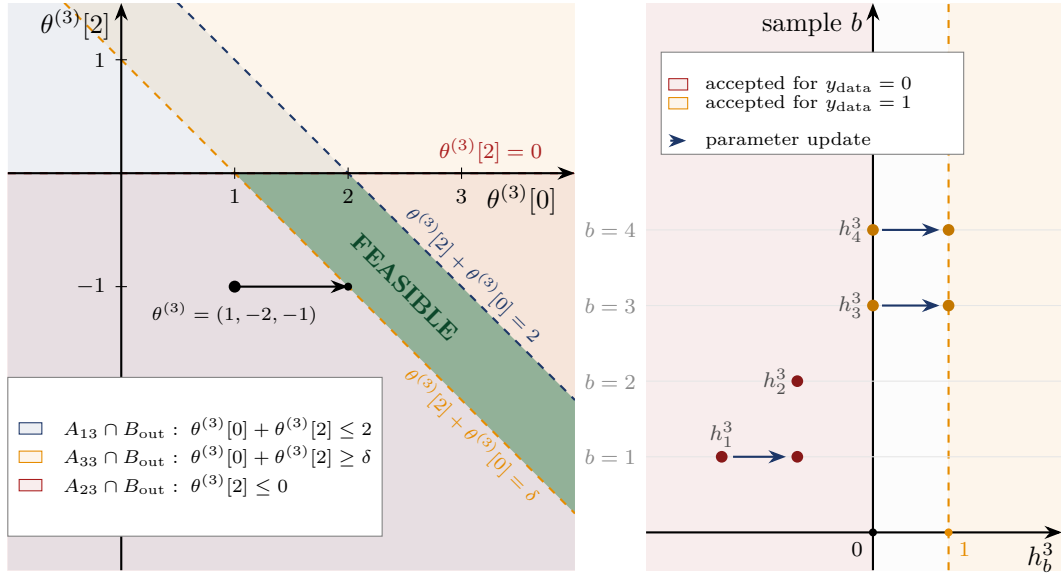


FIGURE 2.6: Combined parameter constraints and output projections for the XOR dataset. The left panel shows the intersection of all constraints on the network parameters, yielding the green feasible region where the network perfectly interpolates the data. The right panel illustrates the corresponding output updates that shift the initial predictions into the accepted regions in the output space.

While this example is artificial, it serves to show how we can move from a set of constraints to a network that perfectly interpolates a dataset. In practice, however, we cannot simply visualize the parameter space and manually select a point in the feasible region. The exact algorithmic approach required to compute this, along with the necessary practical compromises we must make are detailed in the next chapter.

Scalable Projection-Based Training

Existing frameworks [18, 9] partition the constraints in (P) into two disjoint sets and apply the Douglas–Rachford algorithm. This approach has two main limitations, both rooted in the requirement that the projections onto each set be implementable.

First, implementability demands that the constraints contain no coupling between datapoints in a batch. The frameworks achieve this by duplicating the network parameters θ across the batch. While this successfully decouples the per-datapoint constraints, it introduces a substantial memory overhead that constitutes a fundamental computational bottleneck to scaling. Second, the projections require the computation graph to be bipartitioned (i.e., 2-colorable). Forcing this 2-colorability requires inserting identity operation nodes to break edges that would otherwise violate the coloring [9]. In the current implementation of PJAX, this partition is recomputed for each batch, introducing an unnecessary baseline overhead.

To overcome these limitations, we maintain the constraint partition defined in (P) and build upon the method of cyclic projections instead, which successively projects onto each of the sets in a specific cyclic order. We choose an order that corresponds to the PyTorch computational graph: by reinterpreting each functional node in the graph to an associated projection operator, we overwrite the standard backward pass. Consequently, the forward pass remains identical to gradient-based methods, while the backward pass computes a local projection rather than a gradient at each node. The pseudocode for this procedure is detailed in Algorithm 1.

Algorithm 1 Cyclic projections method on the PyTorch computational graph.

```

1:  $h_b^{(N)} \leftarrow y_b \quad b = 1, \dots, B$  ▷ Batch constraint output
2: for  $i = N - 1$  downto 0 do ▷ Traverses the computational graph backwards
3:   for  $b = 1$  upto  $B$  do
4:      $(\bar{h}_b^{(i)}, \bar{\theta}_b^{(i+1)}, \bar{h}_b^{(i+1)}) \in \Pi_{A_{b,i+1}}(h_b^{(i)}, \theta_b^{(i+1)}, h_b^{(i+1)})$ 
5:      $(h_b^{(i)}, \theta_b^{(i+1)}, h_b^{(i+1)}) \leftarrow (\bar{h}_b^{(i)}, \bar{\theta}_b^{(i+1)}, \bar{h}_b^{(i+1)})$  ▷ Architectural constraint
6:   end for
7:    $\bar{\theta}^{(i+1)} \leftarrow \frac{1}{B} \sum_{b=1}^B \theta_b^{(i+1)}$ 
8:    $\theta_b^{(i+1)} \leftarrow \bar{\theta}^{(i+1)} \quad b = 1, \dots, B$  ▷ Consensus constraint
9: end for
10:  $h_b^{(0)} \leftarrow x_b \quad b = 1, \dots, B$  ▷ Batch constraint input

```

Although the projection-based algorithm is now integrated with the PyTorch toolchain, Algorithm 1 still requires duplication of the variables, while also not having performance on par with standard methods on shallow networks. We now describe the key innovations of $\mathcal{P}$ Torch to resolve these issues.

3.1 Scaling the Linear Layer Projection

The linear layer is a ubiquitous component of modern neural network architectures. However, its associated projection currently represents the primary computational bottleneck in projection-based frameworks. Because optimizing this operation is essential for scalability, we introduce two key innovations. First, we propose a memory-efficient ℓ_2 -projection that resolves the memory constraints of existing tools like PJAX. Second, we introduce an ℓ_∞ -projection that, despite a higher computational cost and memory usage, empirically yields better final accuracy.

3.1.1 Fused consensus and bilinear constraint

MOTIVATION. As discussed previously, in [18, 9], the duplication of the network parameters per batch is a primary memory bottleneck. A naive implementation requires constructing an $N_{\text{in}} \times N_{\text{out}} \times B$ tensor, where $N_{\text{in}}, N_{\text{out}}$ are the input and output dimensions of the linear layer respectively, and B is the batch size. This $\mathcal{O}(N_{\text{in}}N_{\text{out}}B)$ memory footprint routinely causes out-of-memory errors on standard hardware, that we also empirically observe in Appendix B.2. The authors of PJAX [9] note:

“the high memory demands of the 4-layer CNN necessitated reducing the number of hidden units per layer to 16 to fit within the available 96GB GPU memory”

By contrast, our benchmarks include a successfully trained CNN with channel widths of 32-64-128 (Section 5.3). Furthermore, we successfully ran a ResNet-8 model, as well as CNNs with 2048 filters. However while they are computationally tractable, deeper networks result in very weak convergence due to the vanishing target problem that we describe in Chapter 4.

DERIVATION. One iteration of the cyclic projections algorithm applied to $\omega \in \Omega$ takes the form $(\Pi_{\tilde{C}_1} \circ \Pi_{\tilde{C}_2} \circ \dots \circ \Pi_{\tilde{C}_N})(\omega)$ on some closed sets $\tilde{C}_1, \dots, \tilde{C}_N$. Because operator composition is associative, we are free to group projections and evaluate them as a single, unified operator. We leverage this property to calculate the fused projection $\Pi_C \circ \Pi_A$ onto the bilinear constraint set A (encoding one row of a matrix-vector product) and the consensus set C without introducing duplicated variables.

Concretely, fix a sample b , layer i , and output neuron j . The weight matrix is $\theta_b^{(i)} \in \mathbb{R}^{N_{\text{out}} \times N_{\text{in}}}$, the input is $h_b^{(i)} \in \mathbb{R}^{N_{\text{in}}}$, and the output is $h_b^{(i+1)} \in \mathbb{R}^{N_{\text{out}}}$. We denote the j th row of $\theta_b^{(i)}$ as $\theta_b^{(i)}[j, :]$, interpreted as a column vector in $\mathbb{R}^{N_{\text{in}}}$, so that $h_b^{(i)}$ and $\theta_b^{(i)}[j, :]$ share the same dimension. The projection onto the bilinear constraint set

that encodes $\theta_b^{(i)}[j, :]^\top h_b^{(i)} = h_b^{(i+1)}[j]$ then requires solving the optimization problem

$$\begin{aligned} & \underset{\bar{h}_b^{(i+1)} \in \mathbb{R}; \bar{h}_b^{(i)}, \bar{\theta}_b^{(i)}[j, :] \in \mathbb{R}^{N_{\text{in}}}}{\text{minimize}} && \|\bar{h}_b^{(i)} - h_b^{(i)}\|_2^2 + \alpha^2 \|\bar{\theta}_b^{(i)}[j, :] - \theta_b^{(i)}[j, :]\|_2^2 + \gamma^2 (\bar{h}_b^{(i+1)} - h_b^{(i+1)})^2 \\ \text{s.t.} &&& \bar{h}_b^{(i+1)} = \bar{\theta}_b^{(i)}[j, :]^\top \bar{h}_b^{(i)} \end{aligned}$$

In this objective, $\alpha, \gamma > 0$ are hyperparameters that reweigh the relative importance of the input, weight, and output terms in the projection. As shown in Appendix D.1, this problem is solved by first computing a scalar t_{jb} via a Newton iteration, where we omitted the superscript (i) for ease of notation. The projected weight then takes the form

$$\bar{\theta}_b^{(i)}[j, :] = u_{jb} \theta_b^{(i)}[j, :] + v_{jb} h_b^{(i)}, \quad \text{where} \quad u_{jb} := \frac{\alpha}{\alpha - t_{jb}^2}, \quad v_{jb} := \frac{t_{jb}}{\alpha - t_{jb}^2}.$$

Next, from Example 2.2, it follows that the projection onto the consensus set forces all weights to agree by taking the batch average. Therefore,

$$\bar{\theta}^{(i)}[j, :] = \frac{1}{B} \sum_{b=1}^B \bar{\theta}_b^{(i)}[j, :] \implies \bar{\theta}^{(i)}[j, :] = \frac{1}{B} \left[\theta^{(i)}[j, :] \sum_{b=1}^B u_{jb} + \sum_{b=1}^B v_{jb} h_b^{(i)} \right].$$

By stacking the scalars u_{jb} and v_{jb} into matrices $U, V \in \mathbb{R}^{N_{\text{out}} \times B}$, and stacking the hidden states into $H \in \mathbb{R}^{B \times N_{\text{in}}}$, we can assemble the update across all output rows simultaneously:

$$\bar{\theta}^{(i)} = \frac{1}{B} \left(\theta^{(i)} \odot (U \mathbf{1}_{B \times N_{\text{in}}}) + V H \right),$$

where $\odot$ denotes the Hadamard product and $\mathbf{1}_{B \times N_{\text{in}}}$ is a $B \times N_{\text{in}}$ matrix of all ones. The second term is evaluated as a single $N_{\text{out}} \times B$ by $B \times N_{\text{in}}$ matrix multiplication. Because these computations are fused, no three-dimensional intermediate tensor is ever formed. As a result, peak memory usage drops from $O(N_{\text{in}} N_{\text{out}} B)$ to $O(N_{\text{in}} B + N_{\text{out}} B + N_{\text{in}} N_{\text{out}})$.

3.1.2 Projection in a different geometry

Motivated by the fact that Adam [27] can be seen as an approximation of steepest descent with respect to the ℓ_∞ -norm [41] and the recently popular optimizer Muon [26] is related to steepest descent under the spectral norm, we also propose an ℓ_∞ variant of the projection onto the bilinear constraint set. The local optimization objective then takes the form

$$\underset{\bar{h} \in \mathbb{R}^{N_{\text{in}}}, \bar{\theta} \in \mathbb{R}^{N_{\text{in}}}, \bar{h}^+ \in \mathbb{R}}{\text{minimize}} \quad \|(\bar{h}, \bar{\theta}, \gamma^2 \bar{h}^+) - (h, \theta, \gamma^2 h^+)\|_\infty \quad \text{s.t.} \quad \bar{\theta}^\top \bar{h} = \bar{h}^+ \quad (3.1)$$

where γ is again a hyperparameter to reweigh the relative importance of input and output in the projection.

ℓ_∞ DERIVATION Throughout this derivation we consider a *single output neuron*, i.e., one row of the weight matrix. Concretely, $\theta, h \in \mathbb{R}^{N_{\text{in}}}$, the pre-activation target $h^+ \in \mathbb{R}$ is a scalar, and the constraint is the scalar bilinear equation $\bar{\theta}^\top \bar{h} = \bar{h}^+$.

The key is finding the smallest feasible box radius $\epsilon^* \geq 0$ such that all constraints are simultaneously satisfiable. Let $\Delta h = \bar{h} - h$ and $\Delta \theta = \bar{\theta} - \theta$, and write $D = \theta^\top h \in \mathbb{R}$, $\bar{D} = \bar{\theta}^\top \bar{h}$ for the current and projected dot products. We search for the optimal ϵ^* found by

$$\underset{\epsilon \geq 0, \bar{h}, \bar{\theta}, \bar{h}^+}{\text{minimize } \epsilon} \quad \text{s.t.} \quad |\Delta h_k| \leq \epsilon, |\Delta \theta_k| \leq \epsilon \quad k \in 1, \dots, N_{\text{in}}, |\bar{h}^+ - h^+| \leq \frac{\epsilon}{\gamma^2}, \bar{\theta}^\top \bar{h} = \bar{h}^+.$$

If $D = \theta^\top h = h^+$ the forward state already satisfies the constraint, so $\epsilon^* = 0$ and the projection is the identity.

Now assume that $D = \theta^\top h < h^+$. We aim to increase D . To this end, note that the inner product is a sum over input coordinates, $D = \sum_{k=1}^{N_{\text{in}}} \theta_k h_k$, so its change splits termwise:

$$\Delta D := \bar{D} - D = \sum_{k=1}^{N_{\text{in}}} (\bar{\theta}_k \bar{h}_k - \theta_k h_k) = \sum_{k=1}^{N_{\text{in}}} \Delta D_k \in \mathbb{R}$$

where $\Delta D_k = \bar{\theta}_k \bar{h}_k - \theta_k h_k$.

Now consider only the k th coordinate. The maximum contribution to ΔD_k happens at one of the corners, so we only need to consider the points in Table 3.1 and Figure 3.1.

TABLE 3.1: Points that maximally change ΔD_k given ϵ

Δh_k	$\Delta \theta_k$	ΔD_k
$+\epsilon$	$+\epsilon$	$\epsilon(h_k + \theta_k) + \epsilon^2$
$-\epsilon$	$-\epsilon$	$-\epsilon(h_k + \theta_k) + \epsilon^2$
$+\epsilon$	$-\epsilon$	$\epsilon(\theta_k - h_k) - \epsilon^2$
$-\epsilon$	$+\epsilon$	$\epsilon(h_k - \theta_k) - \epsilon^2$

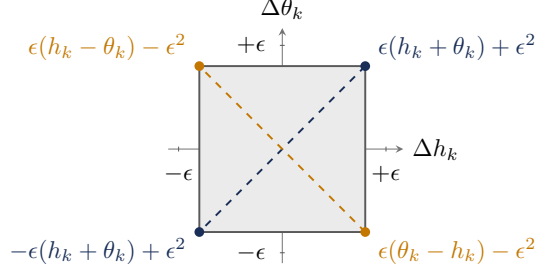


FIGURE 3.1: Graphical interpretation of ΔD_k

The maximum positive contribution is given by

$$M_k(\epsilon) = \max\left(\epsilon|h_k + \theta_k| + \epsilon^2, \epsilon|h_k - \theta_k| - \epsilon^2\right). \quad (3.2)$$

Thus, the smallest feasible radius ϵ^* is the unique positive root of

$$D + \sum_{k=1}^{N_{\text{in}}} M_k(\epsilon) = h^+ - \frac{\epsilon}{\gamma^2} \quad \Longleftrightarrow \quad F(\epsilon) := \sum_{k=1}^{N_{\text{in}}} M_k(\epsilon) + \frac{\epsilon}{\gamma^2} - (h^+ - D) = 0.$$

F is continuous, strictly monotone increasing, and piecewise quadratic in ϵ . Each coordinate contributes at most one kink to F , occurring at

$$\epsilon_k^{\text{kink}} = \max\left\{0, \frac{|h_k - \theta_k| - |h_k + \theta_k|}{2}\right\}.$$

Sorting these kink points requires $\mathcal{O}(N_{\text{in}} \log N_{\text{in}})$ time, with root evaluation linear in N_{in} on each interval.

The remaining case $D > h^+$ follows by symmetry. In particular, the inner product must decrease, which is the previous case under the sign flip $(h^+ - D) \rightarrow |h^+ - D|$ with the displacements reversed. Both branches are captured uniformly by defining $\sigma := \text{sign}(h^+ - \theta^\top h)$, and using the reconstruction

$$\bar{h}^+ = h^+ - \sigma \frac{\epsilon^*}{\gamma^2}, \quad \Delta D_k \text{ taken in the direction of } \sigma.$$

REMOVING THE SORTING PROCEDURE. Maximizing the contribution of coordinate k to ΔD subject to $|\Delta h_k| \leq \epsilon$ and $|\Delta \theta_k| \leq \epsilon$ amounts to maximizing

$$M_k(\epsilon) = \max\{a_k \epsilon + \epsilon^2, b_k \epsilon - \epsilon^2\}, \quad a_k := |h_k + \theta_k|, \quad b_k := |h_k - \theta_k| \quad (3.3)$$

where we performed a change of variables from (h_k, θ_k) to (a_k, b_k) . Geometrically, this transformation is a (scaled) 45 degrees rotation followed by a folding into the nonnegative quadrant, which better matches the geometry of the problem.

The two branches coincide when

$$a_k \epsilon + \epsilon^2 = b_k \epsilon - \epsilon^2 \iff 2\epsilon^2 = (b_k - a_k)\epsilon \iff \epsilon = \frac{1}{2}(b_k - a_k) \quad (\epsilon > 0).$$

For larger ϵ the a_k branch dominates; for smaller ϵ the b_k branch does. The branches only cross in the positive orthant when $b_k > a_k$, i.e. when $h_k \theta_k < 0$; otherwise the a_k branch dominates for all $\epsilon \geq 0$. The unique kink of M_k is therefore

$$\epsilon_k^{\text{kink}} = \max\left\{0, \frac{1}{2}(b_k - a_k)\right\},$$

and we select the active branch with the indicator

$$m_k(\epsilon) := \mathbf{1}[2\epsilon \geq b_k - a_k] \in \{0, 1\},$$

where $m_k = 1$ chooses the equal-sign branch. This gives the branch-free form

$$M_k(\epsilon) = m_k(\epsilon)(a_k \epsilon + \epsilon^2) + (1 - m_k(\epsilon))(b_k \epsilon - \epsilon^2). \quad (3.4)$$

Because this formula for M_k relies on a simple indicator mask rather than conditionals, it can be executed efficiently without any sorting.

F IS CONTINUOUS AND STRICTLY INCREASING. At the kink both branches in (3.3) are equal, so M_k is continuous. It is also non-decreasing on $[0, \infty)$: on the equal-sign branch $M'_k(\epsilon) = a_k + 2\epsilon \geq 0$, and on the opposite-sign branch $M'_k(\epsilon) = b_k - 2\epsilon$, which is active only for $\epsilon \leq \frac{1}{2}(b_k - a_k)$ and thus satisfies $b_k - 2\epsilon \geq a_k \geq 0$. Adding the strictly increasing term ϵ/γ^2 , the residual

$$F(\epsilon) = \sum_{k=1}^{N_{\text{in}}} M_k(\epsilon) + \frac{\epsilon}{\gamma^2} - (h^+ - D)$$

is continuous and strictly increasing, with $F(0) = -(h^+ - D) < 0$ (the case $D < h^+$) and $F(\epsilon) \rightarrow +\infty$ as $\epsilon \rightarrow +\infty$. Hence F has a unique root $\epsilon^* > 0$. Its derivative on each side of every kink is

$$F'(\epsilon) = \sum_{k=1}^{N_{\text{in}}} \left[m_k(\epsilon)(a_k + 2\epsilon) + (1 - m_k(\epsilon))(b_k - 2\epsilon) \right] + \frac{1}{\gamma^2}. \quad (3.5)$$

SOLVING FOR ϵ^* . Both $F(\epsilon)$ and $F'(\epsilon)$ are evaluated in a single $\mathcal{O}(n)$ pass: form a_k, b_k and the masks $m_k(\epsilon)$, then sum. No sorting of kinks and no data-dependent control flow is required. We bracket the root in $[\ell_0, r_0] = [0, \epsilon_{\text{max}}]$ (any ϵ_{max} with $F(\epsilon_{\text{max}}) \geq 0$) and run a safeguarded Newton iteration,

$$\epsilon_{t+1} = \max \left(\ell_t, \min \left(r_t, \epsilon_t - \frac{F(\epsilon_t)}{F'(\epsilon_t)} \right) \right).$$

A bisection safeguard guarantees our Newton iteration converges to ϵ^* . Because F is piecewise quadratic, Newton steps find exact roots within a given piece quickly. Bisection caps the maximum iterations at a constant $\mathcal{O}(\log(1/\delta))$, avoiding $\mathcal{O}(n^2)$ worst-case scenarios. Thus, evaluating F and F' in one pass guarantees $\mathcal{O}(n)$ overall complexity. Empirically, only 5 iterations are needed.

RESULT While this formulation improves final model accuracy, it comes with a computational trade-off. Because the ℓ_∞ update depends non-linearly on the absolute values of individual sample-neuron pairs, it cannot be cleanly factored across the batch dimension like the ℓ_2 case. Consequently, it retains the heavier $\mathcal{O}(N_{\text{in}}N_{\text{out}}B)$ memory footprint. Optimizing this computational bottleneck represents an interesting area for future work, particularly as finding efficient approximations could bridge the gap between this superior accuracy and the memory efficiency required for larger-scale architectures.

3.1.3 Choosing the appropriate geometry

This section highlights a trade-off: we introduce one layer that performs better empirically and another that is more memory efficient, but not one that achieves both simultaneously. Although this hybrid design is conceptually inelegant, there is no practical reason not to mix and match projections. In particular, we can use the memory-efficient projections for large layers with substantial parameter sharing, where memory usage is the primary bottleneck, and use the ℓ_∞ projection for all other layers. This strategy allows us to train arbitrary networks while balancing performance and memory efficiency.

3.2 From Cyclic Projections to a Practical Optimizer

3.2.1 Connection to incremental gradient descent and non-linear relaxation

To motivate the non-linear relaxations (Section 3.2.1), we make the connection between cyclic projections and the incremental gradient method [10, Section 2.4.1].

Recall from Section 2.2 that if the problem (P) is feasible, it is equivalent to the minimization problem

$$\underset{\omega \in \Omega}{\text{minimize}} F(\omega)$$

where

$$F(\omega) := \frac{1}{2} \left\{ \sum_{b=1}^B \sum_{i=1}^N \text{dist}^2(A_{bi}, \omega) + \text{dist}^2(B_{\text{in}}, \omega) + \text{dist}^2(B_{\text{out}}, \omega) + \sum_{i=1}^N \text{dist}^2(C_i, \omega) \right\}$$

which has the form $F(\omega) = \sum_{j=1}^M f_j(\omega)$, with each component $f_j(\omega) = \frac{1}{2} \text{dist}^2(C_j, \omega)$ a squared distance to a constraint set $\tilde{C}_j$. Wherever the projection $\Pi_{\tilde{C}_j}$ is single-valued, f_j is differentiable with

$$\nabla f_j(\omega) = \omega - \Pi_{\tilde{C}_j}(\omega). \quad (3.6)$$

A subtlety is that the constraint sets induced by the network are non-convex, so $\Pi_{\tilde{C}_j}(\omega)$ need not be unique. At points ω that are equidistant from two or more elements of $\tilde{C}_j$, the projection is ambiguous and (3.6) is not well defined. This issue is benign for two reasons. First, the set of such points has Lebesgue measure zero, so (3.6) holds almost everywhere by Rademacher's theorem. Second, on this set the residual $\omega - \Pi_{\tilde{C}_j}(\omega)$ remains a valid subgradient under any choice of $\Pi_{\tilde{C}_j}(\omega)$ in light of [21, Proposition 2.1.10], so the update direction is well-defined in either case. The situation is identical to that of ReLU at the origin: non-differentiable at a single point, yet trained without difficulty using a subgradient selection.

With (3.6) in hand, the cyclic projection update of Algorithm 1 is exactly an incremental gradient step on F . The plain projection step

$$\bar{\theta}^{(i+1)} \leftarrow \Pi_{\tilde{C}_j}(\theta^{i+1}) \quad (3.7)$$

can be rewritten as

$$\bar{\theta}^{(i+1)} \leftarrow (1 - \alpha) \theta^{(i+1)} + \alpha \Pi_{\tilde{C}_j}(\theta^{(i+1)}) \quad \alpha = 1, \quad (3.8)$$

This is a relaxed projection with relaxation factor α . Rearranging (3.8) and substituting (3.6) gives

$$\bar{\theta}^{(i+1)} \leftarrow \theta^{(i+1)} - \alpha(\theta^{(i+1)} - \Pi_{\tilde{C}_j}(\theta^{(i+1)})) = \theta^{(i+1)} - \alpha \nabla f_j(\theta^{(i+1)}), \quad (3.9)$$

which is precisely the incremental gradient update [10, Eq. (2.78)]

$$\omega^{k+1} = \omega^k - \alpha^k \nabla f_{j_k}(\omega^k) \quad (3.10)$$

applied to the finite-sum objective $F(\omega)$, where the step size is α^k and the index j_k is chosen according to the cyclic order induced by the computational graph. This connection motivates the use of first-order optimizers, since the projection residuals we provide are gradients of the finite-sum problem.

NON-LINEAR RELAXATION This last observation is what significantly increases performance. Equation (3.10) is a linear relaxation. But since the residual is a genuine gradient of F , nothing requires the update to be linear in it. Replacing the scaled-residual step with the update rule of any first-order optimizer (e.g Adam) yields a non-linear relaxation, in which the residual is consumed by an adaptive, generally non-linear update.

Because $\mathcal{P}$ Torch already channels projection residuals through Torch’s `autograd`, this requires no algorithmic modification: the residual directly populates the `.grad` attribute, and standard optimizers are invoked exactly as in a conventional training loop (e.g. line 5 of Algorithm 2). In practice, we apply this scheme to the weight residuals $(\bar{\theta}_b^{(i+1)} - \theta_b^{(i+1)})$ to obtain adaptive-learning-rate gradient descent on the weights, while treating the hidden-state residuals as plain projections (equivalent to basic gradient descent, $\alpha = 1$). Using distinct optimization strategies for different parameter groups mirrors the combined use of Muon and Adam [26] in standard gradient-based optimization.

3.2.2 Proximal Projections at the Loss node

The rigid target-assignment step replaces the network’s final prediction h^N with the true target y . We can view this as a projection onto the loss constraint set

$$B_{\text{out}} := \arg \min_u \mathcal{L}(u, y), \quad (3.11)$$

where $\mathcal{L}(\cdot, y)$ is any loss minimized iff its arguments are equal, so $B_{\text{out}} = \{y\}$ and the update is $h^N \leftarrow y$.

The proximal-loss [9] variant relaxes this: rather than snapping to the exact minimizer, it applies the proximal operator of the loss at the current prediction,

$$h^N \leftarrow \bar{h}^N := \text{prox}_{\lambda \mathcal{L}(\cdot, y)}(h^N) \quad (3.12)$$

which we call the proximal target. Recall that for a proper, lower semicontinuous f and parameter $\lambda > 0$, $\text{prox}_{\lambda f}(v) := \arg \min_u \{f(u) + \frac{1}{2\lambda} \|u - v\|^2\}$. Unpacking this with $f = \mathcal{L}(\cdot, y)$ shows the intuition explicitly:

$$\bar{h}^N = \arg \min_u \underbrace{\mathcal{L}(u, y)}_{\text{loss at } u} + \underbrace{\frac{1}{2\lambda} \|u - h^N\|^2}_{\text{stay near } h^N}, \quad (3.13)$$

a λ -controlled compromise between staying close to h^N and reducing $\mathcal{L}(\cdot, y)$, where λ sets both the smoothing strength and the step size. The minimum value attained defines a new function of h^N :

$$\mathcal{L}_\lambda(h^N, y) := \min_u \left\{ \mathcal{L}(u, y) + \frac{1}{2\lambda} \|u - h^N\|^2 \right\}. \quad (3.14)$$

Provided $\mathcal{L}(\cdot, y)$ is proper, lower semicontinuous, and bounded below by an affine function, the objective in (3.14) is coercive for every $\lambda > 0$, so the infimum is attained

and the min is well-defined. This is the Moreau envelope of $\mathcal{L}$ at strength λ . It is a smoothed lower bound: $\mathcal{L}_\lambda \leq \mathcal{L}$ pointwise, $\mathcal{L}_\lambda \rightarrow \mathcal{L}$ as $\lambda \rightarrow 0$, and $\mathcal{L}_\lambda$ is differentiable in h^N even when $\mathcal{L}$ is not. A direct calculation gives the gradient in closed form:

$$\nabla_{h^N} \mathcal{L}_\lambda(h^N, y) = \frac{1}{\lambda} (h^N - \bar{h}^N). \quad (3.15)$$

This is the key identity: the displacement produced by the proximal step, scaled by $1/\lambda$, is the gradient of the smoothed loss. Consequently, the update $h^N \leftarrow \bar{h}^N$ is exactly one gradient-descent step of step size λ on $\mathcal{L}_\lambda(\cdot, y)$:

$$h^N \leftarrow \bar{h}^N = h^N - \lambda \nabla_{h^N} \mathcal{L}_\lambda(h^N, y).$$

Globally, the proximal-loss variant therefore optimizes the surrogate

$$\tilde{F}_\lambda(\omega) = \Phi_{\text{net}}(\omega) + \mathcal{L}_\lambda(h^N(\omega), y), \quad (3.16)$$

where $\Phi_{\text{net}}(\omega)$ collects all internal network terms. Together with Section 3.2.1, this places the loss node on the same footing as every other constraint: the rigid snap $h^N \leftarrow y$ is no longer a special-cased operation but the $\lambda \rightarrow 0$ limit of a genuine gradient step on a single differentiable surrogate.

An important property of this is that the optimum is preserved. For convex $\mathcal{L}(\cdot, y)$, $\mathcal{L}_\lambda$ and $\mathcal{L}$ share the same set of minimizers for every $\lambda > 0$ [34], so choosing λ trades smoothness against surrogate tightness without moving the solution.

Figure B.7 shows a performance improvement under the proximal formulation. We can explain this empirical gain through the bounded backward signal. The residual that initiates the backward pass is $\frac{1}{\lambda}(h^N - \bar{h}^N)$, which for mean-squared-error loss equals $\frac{1}{1+\lambda}(h^N - y)$, i.e. the rigid residual $h^N - y$ contracted by a factor in $(0, 1)$. This contraction is what keeps the backward signal bounded.

This behavior is not special to MSE. Whenever $\mathcal{L}(\cdot, y)$ is convex, the proximal residual is no larger than the rigid (sub)gradient of the loss at h^N , and whenever $\mathcal{L}$ is itself Lipschitz (e.g. hinge, Huber) the residual is bounded uniformly in h^N by the Lipschitz constant of $\mathcal{L}$.

3.2.3 Extension to Mini-Batches and the Full Algorithm

To extend our algorithm to mini-batches, we introduce an inner loop that repeats the backward pass K times for each batch. The hyperparameter K determines how strictly the constraints induced by the current mini-batch are enforced before the algorithm proceeds to the next batch.

This creates a natural optimization trade-off. Larger values of K allow the network to satisfy the current batch constraints more closely, but may lead to overfitting or cause training to stall. Conversely, smaller values of K produce noisier, more stochastic dynamics, as the network switches more rapidly between the potentially conflicting constraints imposed by different mini-batches. This yields the algorithm in Algorithm 2.

Note that the explicit `for` loop over individual samples within a batch has been omitted by using fused operations as described in Section 3.1.1.

Algorithm 2 Mini-Batch $\mathcal{P}$ Torch

```

1:  $h_b^{(N)} \leftarrow \text{prox}_{\lambda\mathcal{L}}(h_b^{(N)}, y_b), \quad b = 1, \dots, B$ 
2: for  $k = 1$  to  $K$  do ▷ Repeat the backward sweep  $K$  times
3:   for  $i = N - 1$  downto  $0$  do ▷ Traverse the computational graph backward
4:      $(\bar{h}^{(i)}, \bar{\theta}^{(i+1)}, \bar{h}^{(i+1)}) \in \Pi_{A_{i+1}}(h^{(i)}, \theta^{(i+1)}, h^{(i+1)})$ 
5:      $(h^{(i)}, \theta^{(i+1)}, h^{(i+1)}) \leftarrow \text{Opt}(h^{(i)} - \bar{h}^{(i)}, \theta^{(i+1)} - \bar{\theta}^{(i+1)}, h^{(i+1)} - \bar{h}^{(i+1)})$ 
6:   end for
7: end for
8:  $h_b^{(0)} \leftarrow x_b$  for  $b = 1, \dots, B$ 

```

In the current $\mathcal{P}$ Torch library, K is fixed to one. This design choice enables a significantly more elegant and efficient implementation: larger values of K require additional memory to propagate the intermediate targets h^+ across multiple backward sweeps and necessitate a non-standard training loop. With $K = 1$, the framework is a near drop-in replacement for standard PyTorch.

3.3 Revisiting the XOR Example: Practical Execution in $\mathcal{P}$ Torch

The introductory example in Section 2.3 established the geometric intuition of data interpolation via projections, it abstracted away the computational graph and how it is implemented. We now build upon this foundation to demonstrate exactly how $\mathcal{P}$ Torch updates the weights of the full network in practice.

3.3.1 Forward and Backward Semantics

For clarity in the following walkthrough, we drop the explicit sample index b from our notation (as introduced in Algorithm 1) and assume the use of the fused projection operators detailed in Section 3.1.1.

A Note on Consensus: In the conceptual XOR example, we applied the architectural and batch constraints sample-by-sample, waiting until the very end to enforce the consensus constraint by finding the intersection of the four slices. In the actual $\mathcal{P}$ Torch implementation, the fused operator executes both the architectural constraints and the consensus. This fusion is what allows the framework to bypass the memory-intensive duplication of network parameters.

3.3.2 Loss Projection and Target Initialization

Let y denote the data target for the batch. The backward pass is initialized by a proximal projection on the loss layer.

The target for the network output $h^{(3)}$ is the proximal step:

$$h^{(3)} = \text{prox}_{\lambda\mathcal{L}}(h^{(3)}, y) \in \arg \min_u \left(\mathcal{L}(u, y) + \frac{1}{2\lambda} \|u - h^{(3)}\|_2^2 \right), \quad (3.17)$$

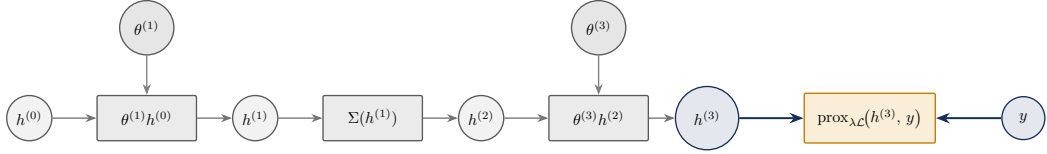


FIGURE 3.2: Initialization of the backward pass: the network output $h^{(3)}$ is coupled to the loss constraint via the proximal operator of Section 3.2.2.

This $h^{(3)}$ is then propagated as the new target output of the network.

In the conceptual XOR example, the proximal operator was effectively a straight projection ($\lambda \rightarrow \infty$), and the loss function was a hard-margin boundary (requiring $h_b^{(3)} \geq \delta$ for positive samples).

3.3.3 Output-Layer Update

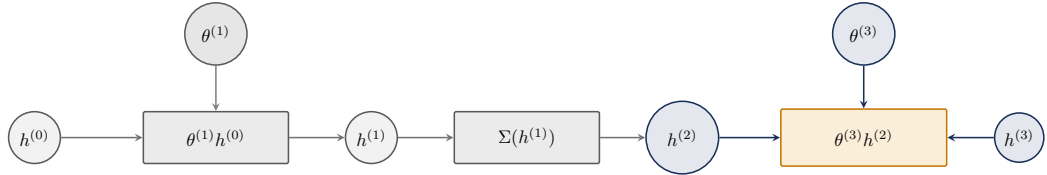


FIGURE 3.3: Update of the output linear layer: projection onto the architectural constraint A_3 couples the output target $h^{(3)}$ to the deepest hidden state $h^{(2)}$ and the weights $\theta^{(3)}$.

In our XOR MLP, layer 3 is the output linear layer with weights $\theta^{(3)}$, so the forward pass behaves as $f_3(h^{(2)}, \theta^{(3)}) = \theta^{(3)}h^{(2)}$. The corresponding architectural constraint set for this specific layer is:

$$A_3 := \{(h^{(2)}, \theta^{(3)}, h^{(3)}) \mid h^{(3)} = \theta^{(3)}h^{(2)}\}. \quad (3.18)$$

Given the current hidden state $h^{(2)}$ and weights $\theta^{(3)}$, together with the output target $h^{(3)}$

$$(\bar{h}^{(2)}, \bar{\theta}^{(3)}, \bar{h}^{(3)}) \in \Pi_{A_3}(h^{(2)}, \theta^{(3)}, h^{(3)}) \quad (3.19)$$

simultaneously yields the target hidden state $\bar{h}^{(2)}$, the projected output weights $\bar{\theta}^{(3)}$, and the adjusted output target $\bar{h}^{(3)}$.

Following the nonlinear relaxation strategy, we do not jump to these exact projected values. Instead, the next parameter iterate is obtained by feeding the projection residual into the chosen first-order optimizer:

$$(h^{(2)}, \theta^{(3)}, h^{(3)}) \leftarrow \text{Optimizer}(h^{(2)} - \bar{h}^{(2)}, \theta^{(3)} - \bar{\theta}^{(3)}, h^{(3)} - \bar{h}^{(3)}; \eta) \quad (3.20)$$

where the step size (or relaxation parameter) η controls how aggressively we adjust the parameters to satisfy this single batch constraint.

3.3.4 Activation function Update

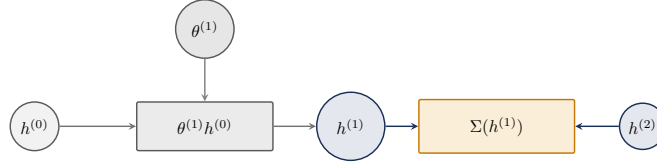


FIGURE 3.4: Projection through the activation constraint A_2 , which carries no parameters.

Now consider the preceding activation layer $f_2 = \Sigma$. As the parameter variable $\theta^{(2)}$ is empty for activation layers, the architectural constraint reduces to a simple relation between two hidden states:

$$A_2 := \{(h^{(1)}, h^{(2)}) \mid h^{(2)} = \Sigma(h^{(1)})\}.$$

The projection simultaneously yields the adjusted target signal for layer 2, $\bar{h}^{(2)}$, and the target signal for layer 1, $\bar{h}^{(1)}$. This $\bar{h}^{(1)}$ then becomes the target passed backward to the preceding linear layer’s projection Π_{A_1} . Note that, depending on the implementation, a non-linear relaxation step may first be applied, similar to the approach used for the weights. This sequential projection procedure repeats, propagating targets backward through the computational graph $\mathcal{G}$ until the root is reached.

3.4 Summary of the $\mathcal{P}$ Torch Algorithm

To give an overview of what we did: we started with plain cyclic projections, chosen because it can be implemented by overriding the PyTorch autograd functions, giving an efficient baseline. We then optimized the linear layer so that it computes the projection without duplicating its parameters, resolving the memory issue present in previous frameworks, introduced an infinity-norm variant that performs better per step empirically. Next, we added a proximal projection at the loss function to improve stability, together with a non-linear relaxation built on the common Adam and Muon optimizers which substantially improve the convergence speed.

Vanishing Target Signals in Deep Networks

A fundamental limitation of projection-based algorithms in deep architectures is the attenuation of the learning signal with depth. We formalize this phenomenon below. At a certain training step, define the target signal magnitude at hidden layer i by

$$\delta^{(i)} := \|\bar{h}^{(i)} - h^{(i)}\|_2.$$

We will operate under the following assumption.

Assumption 4.1 (Local non-expansiveness). *Consider the constraint set associated with the (linear) layer $i + 1$,*

$$A_{i+1} := \{(h^{(i)}, \theta^{(i+1)}, h^{(i+1)}) \mid h^{(i+1)} = f_{i+1}(h^{(i)}, \theta^{(i+1)})\}.$$

Let $P_1 = (h^{(i)}, \theta^{(i+1)}, h^{(i+1)}) \in A_{i+1}$ be the point produced by the forward pass, and let $P_2 = (h^{(i)}, \theta^{(i+1)}, \bar{h}^{(i+1)})$ be the point produced by the backward pass on the next layer. We assume the projection $\Pi_{A_{i+1}}$ satisfies the non-expansiveness property

$$\|\Pi_{A_{i+1}}(P_2) - \Pi_{A_{i+1}}(P_1)\|_2 \leq \|P_2 - P_1\|_2.$$

In general, (global) non-expansiveness only holds for projections onto convex sets, see for example [5, Proposition 4.16]. Nevertheless, this assumption is motivated by empirical evidence in Section 4.2.

Under Assumption 4.1, for the cyclic projection algorithm, the target signal magnitude $\delta^{(i)}$ is monotonically non-increasing as targets are propagated backward from the output layer N . Formally,

$$\delta^{(i)} \leq \delta^{(i+1)} \quad \text{for all } 0 \leq i \leq N - 1,$$

with strict inequality whenever $\|\Delta\theta^{(i+1)}\|_2 > 0$, i.e., whenever the layer is actively learning. The general statement follows by induction; when every layer satisfies $\|\Delta\theta^{(i+1)}\|_2 > 0$, the target magnitudes are strictly decreasing across all layers.

4.1 Proof of vanishing Targets

We argue by downward induction on the layer index i , from the output layer N to the input. For $0 \leq i \leq N$ let

$$\delta^{(i)} < \infty, \quad \text{and if } i < N \text{ then } \delta^{(i)} \leq \delta^{(i+1)}.$$

Establishing the monotone chain $\delta^{(0)} \leq \delta^{(1)} \leq \dots \leq \delta^{(N)}$.

BASE CASE ($i = N$). At the output layer, the proximal update for an arbitrary loss function $\mathcal{L}$ gives

$$\bar{h}^{(N)} \in \arg \min_h \left(\mathcal{L}(h, y_{\text{target}}) + \frac{\lambda}{2} \|h - h^{(N)}\|_2^2 \right).$$

Assuming $\mathcal{L}$ is proper and lower semi-continuous, this minimum is attained and bounded, so $\delta^{(N)} = \|\bar{h}^{(N)} - h^{(N)}\|_2 < \infty$.

INDUCTIVE STEP. Fix $i < N$ and consider the two points in the joint space $(h^{(i)}, \theta^{(i+1)}, h^{(i+1)})$:

$$\begin{aligned} P_1 &= (h^{(i)}, \theta^{(i+1)}, h^{(i+1)}) \\ P_2 &= (h^{(i)}, \theta^{(i+1)}, \bar{h}^{(i+1)}) \end{aligned}$$

Since the forward pass satisfies the constraint exactly, $P_1 \in A_{i+1}$ and hence $\Pi_{A_{i+1}}(P_1) = P_1$. Applying Assumption 4.1 yields

$$\|\Pi_{A_{i+1}}(P_2) - P_1\|_2^2 \leq \|P_2 - P_1\|_2^2.$$

Expanding both sides and using that P_1 and P_2 agree in their first two components,

$$\|\bar{h}^{(i+1)} - h^{(i+1)}\|_2^2 + \|\bar{\theta}^{(i+1)} - \theta^{(i+1)}\|_2^2 + \|\Delta h^{(i+1)}\|_2^2 \leq \|\bar{h}^{(i+1)} - h^{(i+1)}\|_2^2,$$

where $\Delta h^{(i+1)} := h^{(i+1)*} - h^{(i+1)}$ and $h^{(i+1)*}$ is the final value of $h^{(i+1)}$ on the current backward pass. This rearranges to

$$(\delta^{(i)})^2 \leq (\delta^{(i+1)})^2 - (\|\Delta \theta^{(i+1)}\|_2^2 + \|\Delta h^{(i+1)}\|_2^2) \leq (\delta^{(i+1)})^2.$$

By the inductive hypothesis $\delta^{(i+1)} < \infty$, the right-hand side is finite; hence $\delta^{(i)} \leq \delta^{(i+1)} < \infty$, with strict inequality whenever $\|\Delta \theta^{(i+1)}\|_2^2 + \|\Delta h^{(i+1)}\|_2^2 > 0$.

CONCLUSION. By induction $\delta^{(i)} \leq \delta^{(i+1)} < \infty$ holds for all $0 \leq i < N$, giving

$$\delta^{(0)} \leq \delta^{(1)} \leq \dots \leq \delta^{(N)} < \infty.$$

When every active layer satisfies $\|\Delta \theta^{(i+1)}\|_2 > 0$, each inequality is strict and the target magnitudes decrease strictly across all layers. $\square$

Depth Barrier: In a network that is actively learning, cyclic projections result in a diminishing learning signal to early layers. For networks of sufficient depth, the target difference at the earliest layers reaches machine precision, rendering parameter updates in those layers negligible (Figure 5.3).

While no definitive solution to this problem has been found, several workarounds have been explored, including skip connections as in PJAX [9] and applying Muon’s [26] Newton–Schulz orthogonalization, without rescaling, to the hidden states while using a separate optimizer for the weights (Figures 5.4 and 5.5). These heuristics improve performance in deeper networks, largely by stabilizing the target differences across layers. However, performance remains significantly weaker than that of traditional gradient-based methods, making this framework impractical for training deep networks at larger scale.

4.2 Experimental Verification of Local Non-Expansiveness for the Linear Layer

To evaluate Assumption 4.1, we analyze the (P_1, P_2) pairs generated by the training of a 32-layer MLP. During the forward pass, we collect $P_1 = (h^{(i)}, \theta^{(i+1)}, h^{(i+1)})$ along with the target $\bar{h}^{(i+1)}$ to construct P_2 . We then compute the projection $\Pi_{A_{i+1}}(P_2)$ using Newton iteration on the linear layer, and record

$$r^{(i)} := \frac{\|\Pi_{A_{i+1}}(P_2) - P_1\|_2}{\|P_2 - P_1\|_2} = \frac{\|\Pi_{A_{i+1}}(P_2) - P_1\|_2}{\delta^{(i+1)}},$$

which Assumption 4.1 bounds by 1. When $\delta^{(i+1)}$ falls below the solver residual of $\sim 10^{-6}$, $r^{(i)}$ reflects numerical noise rather than geometry; we exclude these records and place them in column n_{vanish} in Table 4.1.

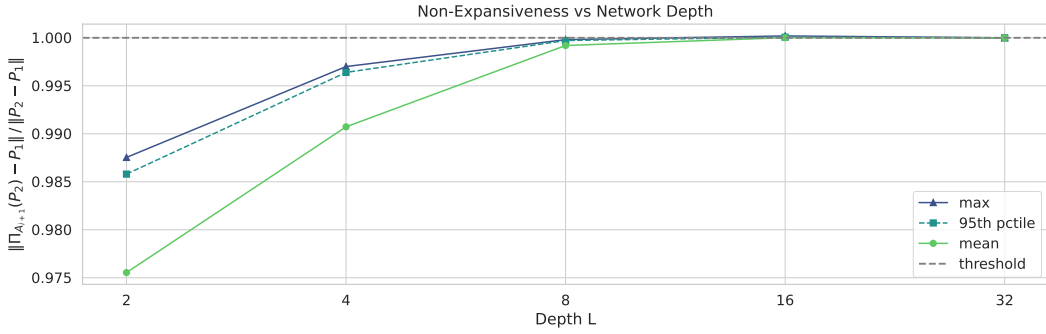


FIGURE 4.1: Progression of the non-expansiveness ratio $r^{(i)}$ across varying network depths L .

The observed ratio can be seen in Figure 4.1. This shows that the equality holds across all depths: $r^{(i)} \leq 1$ up to the solver precision, excluding the single $L = 16$ outlier at 1.0002 whose $\delta^{(i+1)}$ sits just above the cut-off (Table 4.1).

TABLE 4.1: Non-expansiveness ratio $r^{(i)}$ pooled across layers, seeds, and recorded steps, for depths $L \in \{2, 4, 8, 16, 32\}$ (three seeds).

L	n_{valid}	n_{vanish}	$\max r$	$\bar{r}$	$\%\{r > 1 + 10^{-5}\}$
2	18	0	0.9875	0.9755	0%
4	21	13	0.9970	0.9907	0%
8	18	18	0.9998	0.9992	0%
16	18	13	1.0002	1.0000	16.7%
32	9	13	1.0000	1.0000	0%

Experiments

We evaluate $\mathcal{P}$ Torch against Torch [33] as a gradient-based reference implementation and we use PJAX [9] as the projection-based reference. Sections 5.1 and 5.3 benchmark end-to-end accuracy on the standard MNIST and CIFAR-10 classification tasks; Appendices B.1 to B.3 isolate choices specific to $\mathcal{P}$ Torch: proximal loss, efficient matrix-multiplication projection, optimizer choice, relaxation placement and projection norm. Finally, Appendix A investigates hybrid projection gradient-based training and non-differentiable activation functions.

SHARED PROTOCOL. Unless explicitly overridden, all benchmarks share the configuration in Table C.1.

5.1 Shallow MLP: MNIST and CIFAR-10

Setup :

A single-hidden-layer MLP with 512 ReLU units is trained on MNIST ($784 \rightarrow 512 \rightarrow 10$) and CIFAR-10 ($3072 \rightarrow 512 \rightarrow 10$).

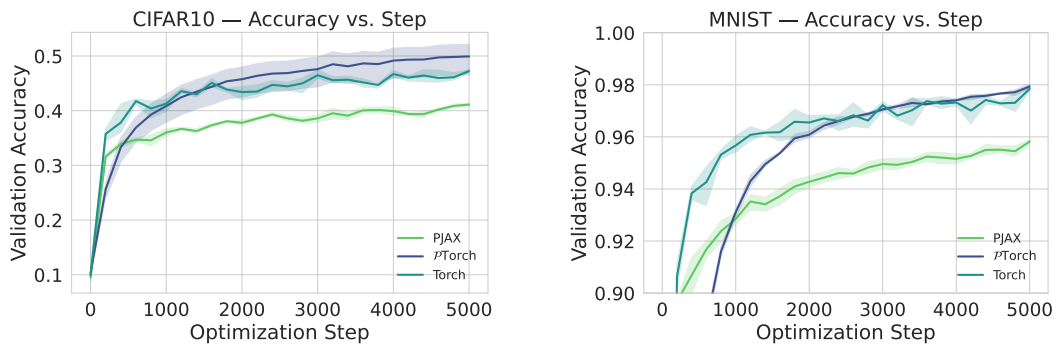


FIGURE 5.1: MLP validation accuracy vs. optimization step for CIFAR-10 (left) and MNIST (right). Shaded bands indicate ± 1 standard deviation.

INTERPRETATION. $\mathcal{P}$ Torch performs comparable to standard gradient-based training. It demonstrates a slight advantage on a per-step basis by reaching higher

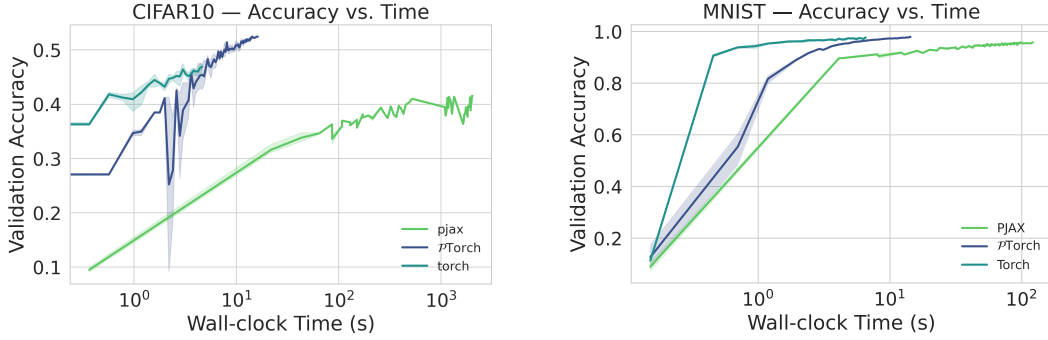


FIGURE 5.2: MLP validation accuracy vs. wall-clock time for CIFAR-10 (left) and MNIST (right).

validation accuracy in fewer optimization steps on CIFAR-10, while its overall execution time remains highly competitive. A logarithmic scale was required for the wall-clock time axis because the PJAX network requires significantly more time to run. Importantly, this evaluation does not claim that $\mathcal{P}$ Torch is superior to Torch for training small MLPs, as the performance gap falls within the variance of standard hyperparameter tuning. Rather, it serves as an empirical proof of concept, demonstrating that projection-based training can be a competitive, practical alternative to established methods for small MLPs.

5.2 Vanishing target signal and depth

Setup :

We train linear MLPs of depth $L \in \{2, 4, 8\}$ and width 32 to recover the identity map $f(x) = x$ on i.i.d. Gaussian inputs $x \sim \mathcal{N}(0, I_{32})$. Each batch contains 16 samples. We log the per-layer target magnitude $\delta_k^n := \frac{\|\bar{h}^{(k),n} - h^{(k),n}\|_2}{B}$ at training step $n = 1$ and the final training MSE.

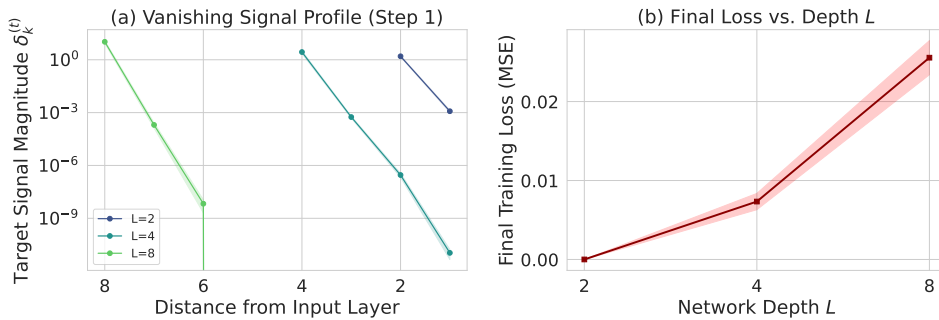


FIGURE 5.3: Target signal magnitude δ_k^n at step 1 for varying depths (left) Final training MSE vs. depth (right) on the identity-recovery task.

5. EXPERIMENTS

INTERPRETATION. The identity-recovery task shows that δ_k^n decays as k moves away from the output, with layers three steps from the output already reaching machine precision. This observation is consistent with the theory in Chapter 4.

Setup :

A 4-layer MLP is trained on MNIST and CIFAR-10 with the protocol of Section 5.1 but with ℓ_2 linear layers. The same architecture in Torch (Adam, $\text{lr} = 10^{-3}$, cross-entropy) serves as the gradient baseline.

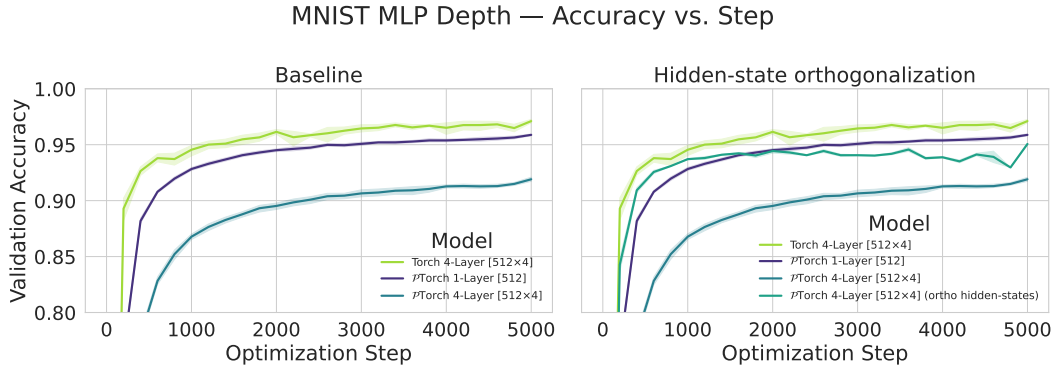


FIGURE 5.4: Validation accuracy vs. optimization steps for a 4-layer MLP on MNIST, comparing runs with and without orthogonalization applied to hidden state update. The performance gap relative to Torch widens with depth, consistent with the depth-barrier prediction of Chapter 4.

INTERPRETATION. Figure 5.4 illustrates in a practical setting what the toy example in Figure 5.3 suggests: optimization difficulty scales with network depth. This creates a counterintuitive dynamic where, unlike gradient-based networks that typically benefit from increased depth, projection-based networks experience degraded performance. As shown on the right of Figure 5.4, orthogonalizing the hidden-state residuals can slightly mitigate this degradation.

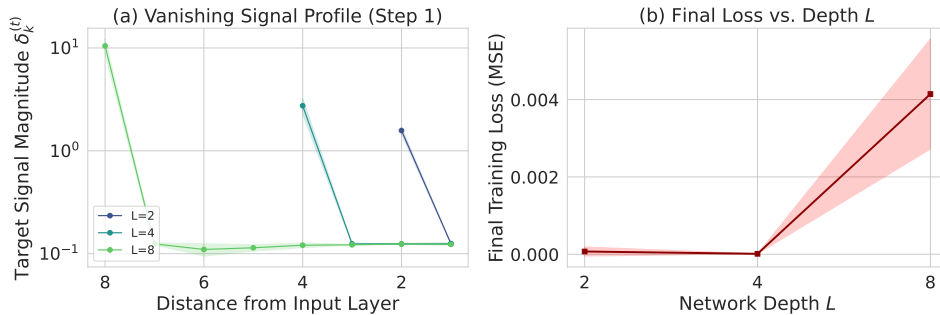


FIGURE 5.5: Applying orthogonalization on the hidden-state residuals stabilises δ_k^n across depths.

This orthogonalization was inspired by the *muon* optimizer and uses the Polar Express orthogonalization procedure [1]. Figure 5.5 repeats the benchmark of Section 5.2 with orthogonalization applied to the residuals, and the stabilised δ_k^n confirms that this artificially counteracts the vanishing targets problem. The approach is not free, however: it adds computational cost, and where the unmodified method benefits from shallowness, orthogonalization slightly degrades performance on shallow networks. Most critically, even with it enabled we have not observed a deep network outperform a shallower variant.

5.3 Convolutional networks: small CNN on CIFAR-10

Setup :

3-layer CNN on CIFAR-10: three blocks with channel widths $\{32, 64, 128\}$ followed by a flatten operation and a linear ℓ_∞ classifier head. Training runs for 10,000 steps. *PTorch* uses *ProjectionAdam* ($\text{lr} = 10^{-3}$).

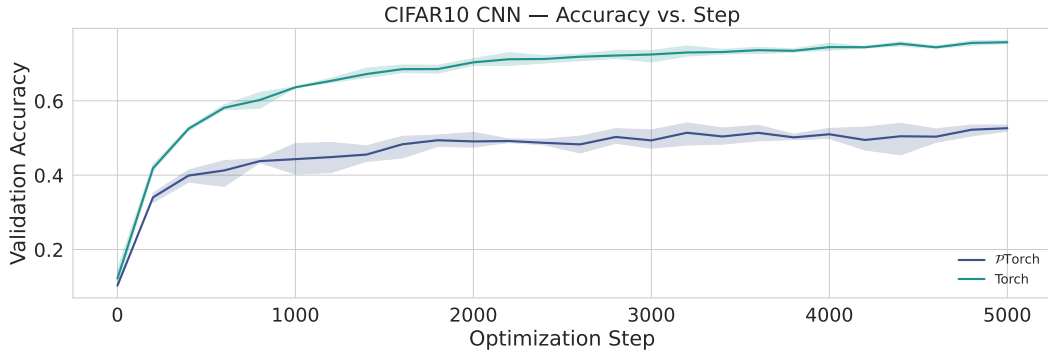


FIGURE 5.6: Validation accuracy vs. wall-clock time on CIFAR-10 with the CNN architecture.

INTERPRETATION. The CNN significantly underperforms its gradient-based counterpart. This performance gap stems from the fundamental architecture of CNNs, which rely heavily on depth to extract hierarchical features from images. Because projection-based learning inherently degrades as network depth increases, this structural mismatch makes training high-performing CNNs particularly challenging. While these results demonstrate that training CNNs is computationally feasible with *PTorch*, they also highlight that substantial progress is necessary before projection-based methods can serve as a viable alternative for standard convolutional architectures.

5.4 Training a ViT

We now examine what happens when we train a modern deep architecture by evaluating a small Vision Transformer (ViT) on CIFAR-10.

5. EXPERIMENTS

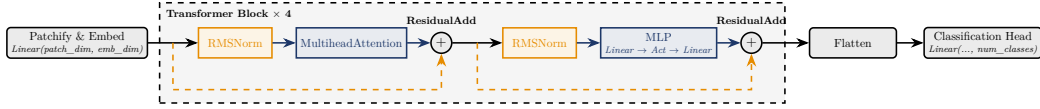


FIGURE 5.7: Architecture of the small ViT used for the CIFAR-10 experiment.

IMPLEMENTING RESIDUAL BLOCKS. The residual architecture introduces two new operators. First, the branch operator clones an input n times. We cannot simply copy the tensors because, by default, autograd sums the incoming targets instead of averaging them as required. Second, the add operator, $z = h_{\text{skip}} + h_{\text{out}}$, projects a target by distributing the displacement $\delta = z^* - z$ equally between the two operands.

Setup :

A small ViT with 4 transformer blocks, a patch size of 4 and 16 attention heads. ℓ_2 projection and a batch size of 256.

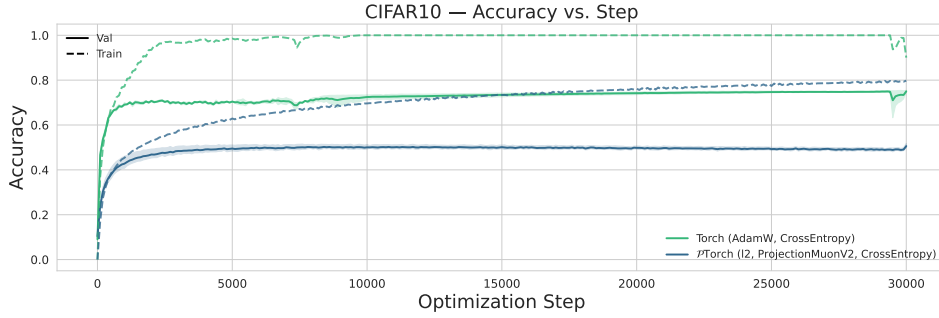


FIGURE 5.8: Train and validation accuracy per step for the ViT on CIFAR-10.

INTERPRETING THE RESULTS. Considered in isolation, the training accuracy of $\sim 80\%$ appears encouraging; validation accuracy, however, plateaus near $\sim 50\%$, matching the level of the shallow MLP and remaining well below that of a gradient-trained ViT. Introducing regularization or data augmentation reduces training accuracy to $\sim 60\%$. An untuned gradient-based baseline is also provided. It highlights the comparative weakness of the projection-based optimizer by demonstrating that gradient descent yields substantially better results.

LIMITATIONS. The reported ViT figures should be regarded as optimistic. They were obtained through an automated tuning procedure in which numerous configurations were executed and diagnosed under the guidance of a reasoning LLM until one converged, and they therefore do not reflect the performance that an arbitrary architecture would achieve without such tuning. Although the ability to train a transformer at all is a noteworthy outcome, the approach in its present form remains far from a compute-efficient alternative to gradient-based training, and we regard it primarily as a proof of computational tractability rather than as a practical method.

Beyond $\mathcal{P}$ Torch

$\mathcal{P}$ Torch demonstrates that projection-based learning can be competitive with gradient-based methods on shallow architectures. By building natively on PyTorch’s autograd engine, it removes prior memory bottlenecks and connects cyclic projections to first-order optimization through nonlinear relaxation.

However, two major obstacles remain. The first is depth: our vanishing-target theorem shows that the learning signal attenuates with network depth, creating a challenge structurally analogous to the vanishing-gradient problem that limited early deep learning before the advent of residual connections and principled initialization schemes. The second is coverage: effective projections for many task-specific architectures have yet to be explored and implemented.

Neither barrier is likely to fall in a single paper. Gradient-based deep learning succeeded not because every early idea worked, but because enough ideas were explored for the successful ones to accumulate into a practical toolkit. Projection-based learning may require a similar period of experimentation. In that setting, $\mathcal{P}$ Torch provides both a practical research platform and a competitive baseline: its projections can already improve the existing system (PJAX), while its native integration with PyTorch enables hybrid architectures and direct comparison with standard optimization pipelines.

Appendices

Advanced Architectural Evaluations

A.1 Hybrid architecture

To demonstrate the flexibility and interoperability of $\mathcal{P}$ Torch, we evaluate a hybrid architecture that combines projection-based and gradient-based layers within a computational graph.

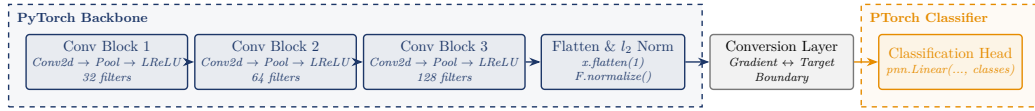


FIGURE A.1: Architecture of the CNN with a Torch based feature extractor and $\mathcal{P}$ Torch head.

Setup :

We construct a hybrid Convolutional Neural Network (CNN) (Figure A.1) for the MNIST and CIFAR-10 image classification tasks, comprising a standard PyTorch backbone and a $\mathcal{P}$ Torch classification head.

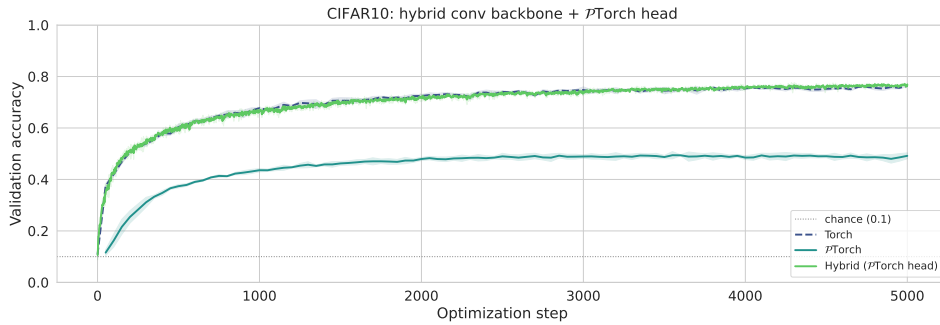


FIGURE A.2: Training dynamics of the hybrid CNN architecture. The integration of $\mathcal{P}$ Torch projection modules with standard PyTorch gradient-based layers demonstrates that hybrid architectures can be trained effectively.

This architectural interoperability represents a highly promising capability of $\mathcal{P}$ Torch. We can optimize a deep feature extractor using standard gradients while natively enforcing rigid constraints at the output layer via projections. Consequently, this hybrid paradigm allows us to bypass the chain rule entirely for specialized components. An example application could be the integration of a sorting output head (e.g., using the PAVA projection [11]), directly into an otherwise gradient-optimized network.

A.2 Hybrid optimization

Because the projection / gradient distinction is toggled by a single flag, we can train a network with both signals sequentially: K projection steps, then K gradient steps, and so on. This is useful when an architecture contains non-differentiable components that are merely approximated in the gradient regime but can be optimized exactly through projections. We provide a proof of concept: the goal is to show that the two regimes can be interleaved within a single training run without destabilizing optimization, not to demonstrate a regime in which the hybrid schedule outperforms either training mode in isolation.

Setup :

We use a network similar to Figure A.2 but with RMS-normalization layers inserted between every conv block, and a $\mathcal{P}$ Torch backbone in place of the gradient one.

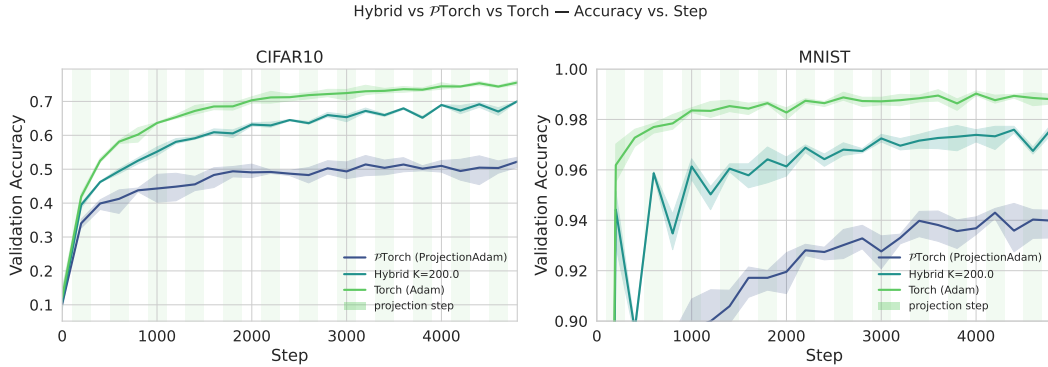
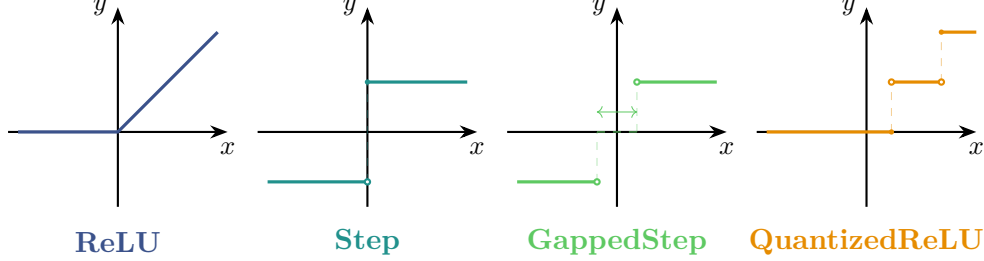


FIGURE A.3: Training dynamics on CIFAR-10 (left) and MNIST (right) under hybrid optimization.

A.3 Non-differentiable activation functions

Gradient-based optimization imposes a strict assumption: every component of the computational graph must admit a tractable subgradient, projection-based

optimization bypasses this limitation. To demonstrate this capability, we evaluate $\mathcal{P}$ Torch classification’s capacity to optimize architectures containing fundamentally non-differentiable or discontinuous activation functions.



Setup :

We train a three-hidden-layer multilayer perceptron of width 256 on the MNIST dataset ($784 \rightarrow 256 \rightarrow 256 \rightarrow 256 \rightarrow 10$), training the architecture for 2,000 steps utilizing a batch size of 512. We evaluate a suite of activation functions: a continuous ReLU baseline, alongside Step, GappedStep, and QuantizedReLU.

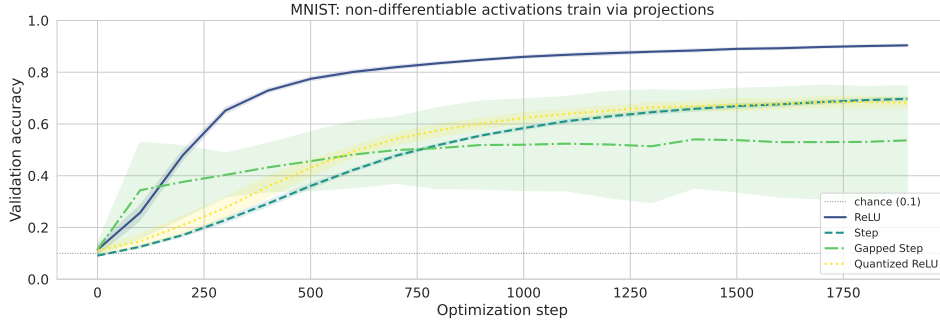


FIGURE A.4: Validation accuracy versus optimization step for networks equipped with continuous and discontinuous activation functions evaluated on MNIST.

This experiment demonstrates that training these functions natively in $\mathcal{P}$ Torch is feasible. As expected, they underperform compared to ReLU. Their performance can be improved by tuning the step/gap size to effectively make them less discontinuous. In gradient-based frameworks, it is practically possible to train networks with these activation functions; however, they require heuristics such as the straight-through estimator to do so. Straight-through estimators [8] work by using the true function on the forward pass and a proxy continuous variant on the backward pass, thus bypassing the non-differentiability issue.

Practical Considerations for Projection-Based Training

B.1 Architecture and Initialization

Shallow networks are preferred in this framework, being both more stable and empirically stronger than their deeper counterparts. Equally important is maintaining a clean computational graph: every operation must either leave gradients untouched under standard Torch or be replaced by a $\mathcal{P}$ Torch equivalent. Mixing in standard Torch layers confuses targets with gradients in ways that are easy to miss. The resulting bugs are often silent; their effects only become apparent as instabilities during training

Initialization is critical as can be seen in Figure B.1; at present, the framework relies on the same initialization heuristics as commonly used in gradient-based training. This is motivated by the connection of our method with incremental gradient descent (Section 3.2.1). However, there has been limited research into better initialization schemes, and this can thus be an area of significant improvement.

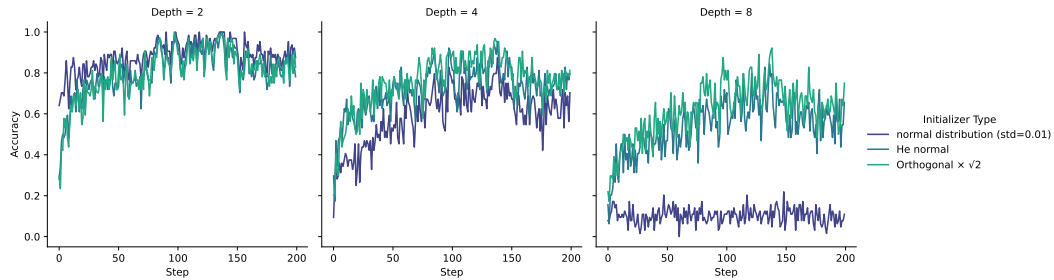


FIGURE B.1: Impact of initialization strategies on the network convergence across varying depths. Training accuracy is shown over 200 steps for networks of depth 2, 4, and 8, comparing three initialization methods: standard normal distribution (std=0.01), He normal, and scaled orthogonal. While all evaluated methods achieve convergence in shallow networks (Depth = 2), the normal distribution initialization degrades significantly as depth increases, failing to converge at Depth = 8.

B.2 Optimizer choice

Choosing the optimizer is one of the most important practical decisions when training a model. In most settings, the Muon optimizer yields the strongest results and is therefore our recommended default (Figure B.2). If faster initial progress is needed, or if Muon diverges, SGD is a robust alternative. When using SGD or Adadelta, higher learning rates sometimes as large as 10 are best. Muon and Adam generally perform best at their standard learning rates of $\sim 10^{-3}$, SGD at a learning rate of ~ 1 .

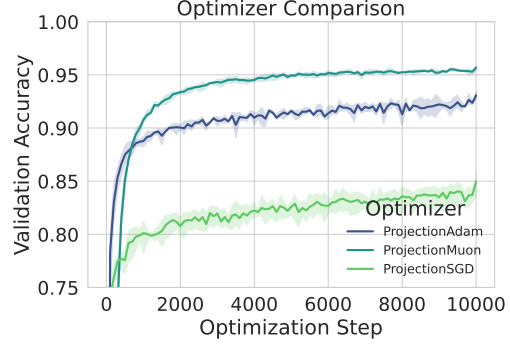


FIGURE B.2: Comparison of different optimizers on the MNIST benchmark.

B.2.1 Projection Mechanics

The two projection norms induce meaningfully different training dynamics. In terms of accuracy and per-step convergence speed, ℓ_∞ is the stronger choice. Its weakness is memory: the footprint scales as $\mathcal{O}(N_{\text{in}} \times N_{\text{out}} \times B)$, on par with PJAX. This batch-size dependence creates a crossover effect in wall-clock time visible in Figure B.3: ℓ_2 runs faster at large B , but is eventually overtaken as optimization progresses.

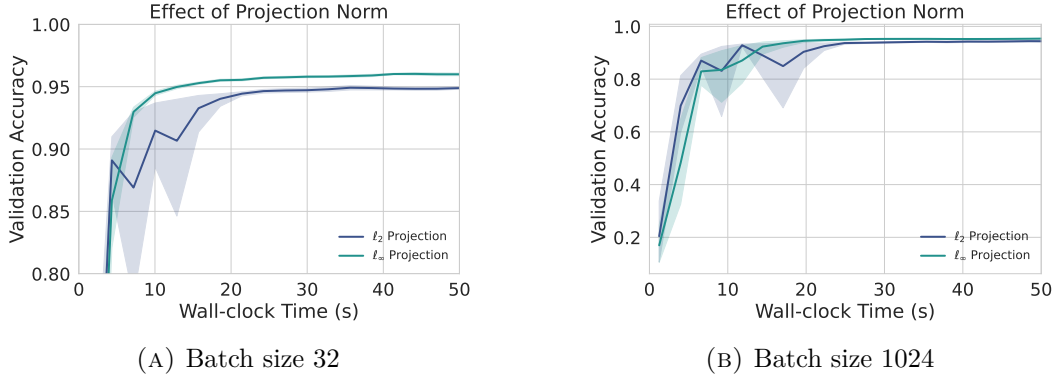


FIGURE B.3: Effect of projection norm on validation accuracy over wall-clock time.

Empirically, both standard PyTorch and $\mathcal{P}$ Torch (configured with an ℓ_2 projection) scale significantly better and maintain a lower memory baseline than PJAX (Figure B.4), enabling the training of substantially larger networks before memory exhaustion. This advantage stems from a fundamental difference in asymptotic complexity. $\mathcal{P}$ Torch reduces the overhead to $\mathcal{O}(N_{\text{in}}B + N_{\text{out}}B + N_{\text{in}}N_{\text{out}})$, matching the standard asymptotic scaling of PyTorch.

B. PRACTICAL CONSIDERATIONS FOR PROJECTION-BASED TRAINING

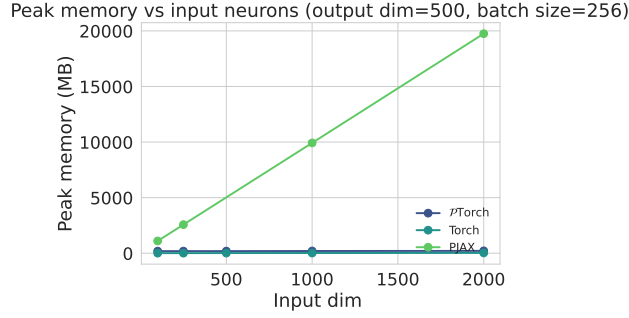
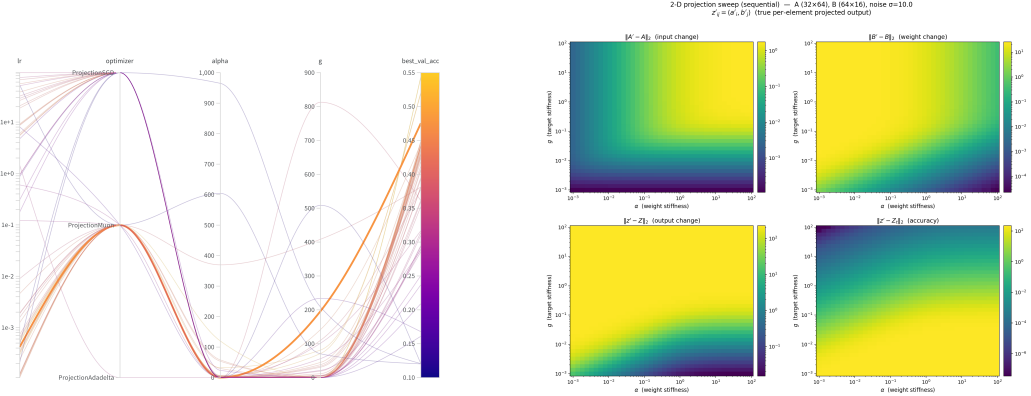


FIGURE B.4: Peak memory versus input dimension for a linear-layer. PJAX scales substantially worse than $\mathcal{P}$ Torch and Torch. For small input dimensions, $\mathcal{P}$ Torch uses roughly $10\times$ more memory than Torch, but this gap narrows as the input dimension increases.

To get better performance the projections can be fine tuned via the α and γ parameters, which dictate whether the weights or activations absorb the majority of the update (Figure B.5b). Specifically, setting $\gamma \rightarrow \infty$ leaves the output values unchanged, whereas setting $\alpha \rightarrow \infty$ leaves the weights unchanged. In practice,



(A) Hyperparameter sweep illustrating the sensitivity to α and γ .

(B) MatMul projection showing the impact of the tuning parameters.

the framework is highly robust and relatively insensitive to these hyperparameters (Figure B.5a); however, careful tuning within a reasonable range, typically $[0.01, 500]$, can yield modest gains when maximizing accuracy.

Another tuning parameter for the cyclic projection algorithm is the number of backward passes per data batch. In the current framework, this is fixed at 1, as it allows for a substantially more efficient and elegant implementation. However, this choice may not be optimal, since using multiple backward passes can improve performance per step (Figure B.6).

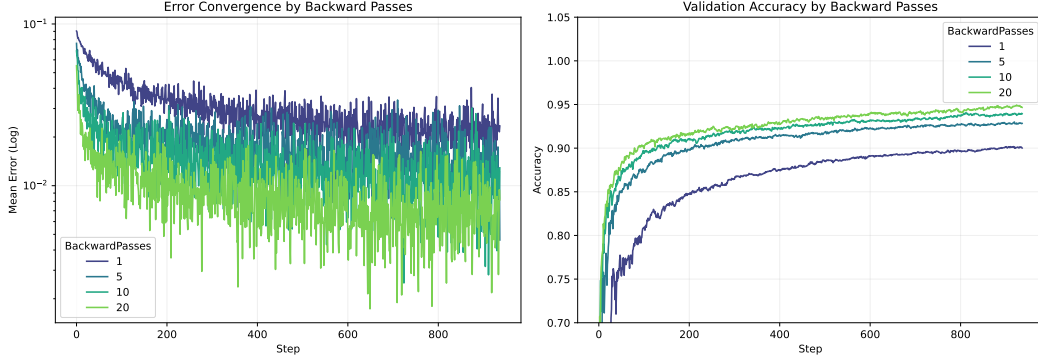


FIGURE B.6: Error convergence and validation accuracy as a function of the number of backward passes on the MNIST benchmark.

B.3 Loss Formulation and Advanced Relaxations

Regarding the loss formulation, replacing the hard target-assignment with a proximal-loss step (Section 3.2.2) accelerates convergence (Figure B.7). Similar benefits to softening the loss constraint have also been reported in prior projection-based literature like PJAX [9].

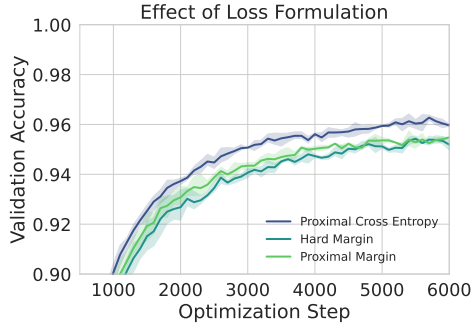


FIGURE B.7: Replacing hard target-assignment with a proximal-loss step accelerates convergence.

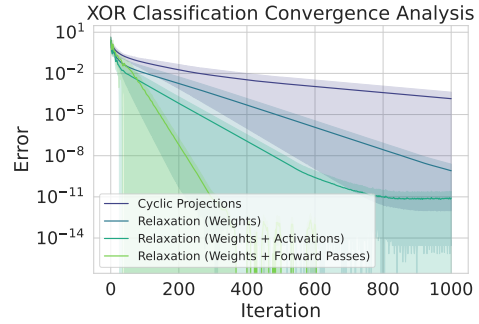


FIGURE B.8: First-order relaxation applied to the hidden states on the XOR benchmark.

To further stabilize training, first-order relaxations can also be applied to the projection residuals. Where this relaxation is applied is critical: applying it to both the weight residuals and the hidden states can perform better than applying it to only the weights. Doing this comes with a large memory penalty, as both the hidden states and the weights need to be tracked. To keep the framework simple, we apply relaxation only to the weights by default. However, in some cases, such as the XOR benchmark, applying relaxation more broadly is beneficial (Figure B.8).

Details Experiments

Unless overridden in a per-experiment paragraph, all benchmarks share the configuration in Table C.1. The configurations underlying every figure is released alongside the code in `experiments/{mlp,deep,cnn_benchmarks,attention}/*.yaml`; the entry points are `bench_ptorch.py`, `bench_torch.py`

TABLE C.1: Default experimental protocol. Per-experiment overrides (maximum step budget, early-stopping patience, network width/depth, loss function, projection norm, and optimizer) are stated in the corresponding paragraphs and reflect the released YAML configs.

Component	Default value
Frameworks compared	$\mathcal{P}$ Torch, PJAX [9], Torch [33]
Random seed	43 (per-run seed = $43 + r$ for $r = 0, 1, 2$)
Batch size	32
Evaluation interval	every 200 optimization steps on the held-out validation split
Train/val	90%/10% of the train set;
Initialization	Kaiming-normal
$\mathcal{P}$ Torch optimizer	ProjectionMuon , lr = 10^{-3}
$\mathcal{P}$ Torch projection norm	ℓ_∞
$\mathcal{P}$ Torch projection scales	$\alpha = \gamma = 1$ (default)
$\mathcal{P}$ Torch loss	CrossEntropy
PJAX optimizer	DouglasRachford , 50 inner projection steps per batch
Torch optimizer	Adam , lr = 10^{-3} , $\beta_1=0.9$, $\beta_2=0.999$
Torch loss	CrossEntropyLoss
Hardware / software	1× NVIDIA L40S (46 GB); 16 cores; 188 GB RAM

Useful Projections

D.1 Linear layers

Linear Layer : Constraint Set

$$C_{\text{dot}} = \{(h, \theta, h^+) \mid \theta^\top h = h^+\}$$

Projection ℓ_2 ($\alpha = 1$)

$$\begin{aligned}\bar{h} &= \frac{h + t\theta}{1 - t^2}, \\ \bar{\theta} &= \frac{\theta + t h}{1 - t^2}, \\ \bar{h}^+ &= h^+ - \frac{t}{\gamma^2}\end{aligned}$$

Derivation Source [6, 18, 9] *The scalar t is computed via Newton's method.*

Projection ℓ_∞

$$\sigma := \text{sign}(h^+ - \theta^\top h), \quad a_k = |h_k + \theta_k|, \quad b_k = |h_k - \theta_k|.$$

The optimal box radius ϵ^* is the unique root of $F(\epsilon) = 0$:

$$F(\epsilon) = \sum_{k=1}^n M_k(\epsilon) + \frac{\epsilon}{\gamma^2} - |h^+ - \theta^\top h|,$$

$$M_k(\epsilon) = m_k(\epsilon)(a_k\epsilon + \sigma\epsilon^2) + (1 - m_k(\epsilon))(b_k\epsilon - \sigma\epsilon^2), \quad m_k(\epsilon) = \mathbf{1}[2\sigma\epsilon \geq b_k - a_k].$$

The projected coordinates are reconstructed as

$$\bar{h}^+ = h^+ - \sigma \frac{\epsilon^*}{\gamma^2}, \quad \begin{cases} \Delta h_k = \Delta \theta_k = \sigma \text{sgn}(h_k + \theta_k) \epsilon^* & m_k = 1 \\ \Delta \theta_k = -\Delta h_k = \sigma \text{sgn}(h_k - \theta_k) \epsilon^* & m_k = 0 \end{cases}$$

ℓ_2 DERIVATION We are looking for the closest point in the feasibility set under a weighted ℓ_2 norm:

$$\Pi_{C_{\text{dot}}}^{\alpha, \gamma}(h, \theta, h^+) \in \arg \min_{(\bar{h}, \bar{\theta}, \bar{h}^+) \in C_{\text{dot}}} \left(\|\bar{h} - h\|^2 + \alpha \|\bar{\theta} - \theta\|^2 + \gamma^2 (\bar{h}^+ - h^+)^2 \right).$$

Introduce a Lagrangian multiplier λ and define

$$\mathcal{L} = \|\bar{h} - h\|^2 + \alpha \|\bar{\theta} - \theta\|^2 + \gamma^2 (\bar{h}^+ - h^+)^2 + \lambda (\bar{h}^+ - \bar{\theta}^\top \bar{h}).$$

$$\nabla \mathcal{L} = 0 \implies \bar{h} = h + t \bar{\theta}, \quad \bar{\theta} = \theta + \frac{t}{\alpha} \bar{h}, \quad \bar{h}^+ = h^+ - \frac{t}{\gamma^2} \quad (t := \lambda/2)$$

Solving the coupled system gives

$$\bar{h} = \frac{\alpha}{\alpha - t^2} (h + t \theta), \quad \bar{\theta} = \frac{\alpha \theta}{\alpha - t^2} + \frac{t h}{\alpha - t^2}.$$

Substituting $\bar{h}^+ = \bar{\theta}^\top \bar{h}$ into the constraint leads to a scalar equation $f(t) = 0$:

$$h^+ - \frac{t}{\gamma^2} = \frac{(\alpha \theta + t h)^\top \alpha (h + t \theta)}{(\alpha - t^2)^2} = \alpha \frac{p(\alpha + t^2) + q t}{(\alpha - t^2)^2},$$

where $p := \theta^\top h$ and $q := \|h\|^2 + \alpha \|\theta\|^2$. The root can be found efficiently with Newton's method, after which $\bar{h}, \bar{\theta}, \bar{h}^+$ are reconstructed from the expressions above.

D.2 Functional layers

Add Layer : Constraint

$$C_{\text{add}} = \{(h_1, h_2, h^+) \mid h_1 + h_2 = h^+\}$$

Projection ℓ_2

Let the residual be $r = h_1 + h_2 - h^+$. The displacement is shared in thirds:

$$\begin{aligned} \bar{h}_1 &= h_1 - \frac{r}{3}, \\ \bar{h}_2 &= h_2 - \frac{r}{3}, \\ \bar{h}^+ &= h^+ + \frac{r}{3}. \end{aligned}$$

If the output target is fixed, projecting (h_1, h_2) onto the constraint $h_1 + h_2 = h^+$ splits this residual evenly between the two branches

$$\bar{h}_1 = h_1 - \frac{r}{2}, \quad \bar{h}_2 = h_2 - \frac{r}{2}.$$

Branch Layer :**Constraint**

$$C_{\text{copy}} = \{(h, h_1^+, \dots, h_N^+) \mid h_1^+ = h_2^+ = \dots = h_N^+ = h\}$$

Projection ℓ_2

all N copies must agree with the source.

$$(\bar{h} - h)^2 + \sum_{i=1}^N (\bar{h}_i^+ - h_i^+)^2 \quad \text{s.t.} \quad \bar{h}_i^+ = \bar{h}$$

gives the average of all $N + 1$ coordinates

$$\bar{h} = \bar{h}_i^+ = \frac{h + \sum_{i=1}^N h_i^+}{N + 1}.$$

D.3 Activation functions**Softmax (heuristic) :**

Unlike the other projections, this is not the joint projection: we project onto the two sides of the constraint sequentially.

Constraint

$$C_{\text{softmax}} = \left\{ (h, h^+) \in \mathbb{R}^n \times \mathbb{R}^n \mid h_i^+ = \frac{e^{h_i}}{\sum_j e^{h_j}} \right\}$$

Feasible outputs satisfy $h_i^+ > 0$ and $\sum_i h_i^+ = 1$, and the input is determined only up to a constant shift: h and $h + c\mathbf{1}$ produce the same h^+ .

Sequential projection

A target h^+ is projected onto the simplex by clamping to positivity and re normalizing:

$$\bar{h}^+ = \frac{\max(h^+, \varepsilon)}{\sum_j \max(h_j^+, \varepsilon)}.$$

By softmax's shift invariance, any feasible input has the form $\log \bar{h}^+ + c\mathbf{1}$ for some scalar c . Choosing c to minimize $\|\bar{h} - h\|_2^2$ projects h onto that ray:

$$c = \frac{1}{n} \sum_{i=1}^n (h_i - \log \bar{h}_i^+) = \text{mean}(h - \log \bar{h}^+), \quad \bar{h} = \log \bar{h}^+ + c\mathbf{1}.$$

ReLU Layer (ℓ_2) :
Constraint

$$C_{\text{ReLU}} = \{(h, h^+) \mid h^+ = \max(0, h)\}$$

Projection ℓ_2

Let the projection onto the inactive branch be $h_1 = \min(h, 0)$:

$$d_1^2 = (h - h_1)^2 + (h^+)^2$$

Let the projection onto the active branch be $h_2 = \max\left(\frac{h+h^+}{2}, 0\right)$:

$$d_2^2 = (h - h_2)^2 + (h^+ - h_2)^2$$

The projected coordinates $(\bar{h}, \bar{h}^+)$:

$$(\bar{h}, \bar{h}^+) = \begin{cases} (h_1, 0) & \text{if } d_1^2 < d_2^2 \\ (h_2, h_2) & \text{otherwise} \end{cases}$$

Derivation Source [18]

ReLU Layer (ℓ_∞) :
Constraint

$$C_{\text{ReLU}} = \{(h, h^+) \mid h^+ = \max(0, h)\}$$

Projection ℓ_∞

Let the projection onto the inactive branch be $h_1 = \min(h, 0)$:

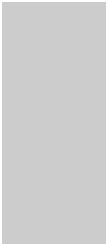
$$d_1 = \max(|h - h_1|, |h^+|)$$

Let the projection onto the active branch be $h_2 = \max\left(\frac{h+h^+}{2}, 0\right)$:

$$d_2 = \max(|h - h_2|, |h^+ - h_2|)$$

The projected coordinates $(\bar{h}, \bar{h}^+)$:

$$(\bar{h}, \bar{h}^+) = \begin{cases} (h_1, 0) & \text{if } d_1 < d_2 \\ (h_2, h_2) & \text{otherwise} \end{cases}$$



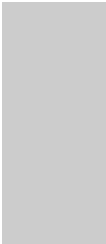
Bibliography

- [1] N. Amsel, D. Persson, C. Musco, and R. M. Gower. The polar express: Optimal matrix sign methods and their application to the Muon algorithm. *arXiv preprint arXiv:2505.16932*, 2025.
- [2] H. Attouch, J. Bolte, and B. F. Svaiter. Convergence of descent methods for semi-algebraic and tame problems: proximal algorithms, forward–backward splitting, and regularized Gauss–Seidel methods. *Mathematical programming*, 137(1):91–129, 2013.
- [3] A. Auslender. *Méthodes numériques pour la résolution des problèmes d’optimisation avec contraintes*. PhD thesis, Faculté des Sciences, Grenoble, 1969.
- [4] H. H. Bauschke. *Projection algorithms and monotone operators*. PhD thesis, Simon Fraser University, 1996.
- [5] H. H. Bauschke and P. L. Combettes. *Convex analysis and monotone operator theory in Hilbert spaces*. Springer, 2020.
- [6] H. H. Bauschke, M. K. Lal, and X. Wang. Projecting onto rectangular hyperbolic paraboloids in Hilbert space. *Applied Set-Valued Analysis and Optimization*, 5(2), 2023.
- [7] H. H. Bauschke and W. M. Moursi. On the Douglas–Rachford algorithm for solving possibly inconsistent optimization problems. *Mathematics of Operations Research*, 49(1):58–77, 2024.
- [8] Y. Bengio, N. Léonard, and A. Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.
- [9] A. Bergmeister, M. K. Lal, S. Jegelka, and S. Sra. A projection-based framework for gradient-free and parallel learning. *arXiv preprint arXiv:2506.05878*, 2025.
- [10] D. P. Bertsekas. Nonlinear programming. *Journal of the Operational Research Society*, 48(3):334–334, 1997.

- [11] M. J. Best and N. Chakravarti. Active set algorithms for isotonic regression; a unifying framework. *Mathematical Programming*, 47(1):425–439, 1990.
- [12] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, et al. Jax: composable transformations of python+ numpy programs, 2018.
- [13] L. M. Bregman. The relaxation method of finding the common point of convex sets and its application to the solution of problems in convex programming. *USSR Computational Mathematics and Mathematical Physics*, 7(3):200–217, 1967.
- [14] M. Carreira-Perpinan and W. Wang. Distributed optimization of deeply nested systems. In *Artificial intelligence and statistics*, pages 10–19, 2014.
- [15] J. Douglas and H. H. Rachford. On the numerical solution of heat conduction problems in two and three space variables. *Transactions of the American mathematical Society*, 82(2):421–439, 1956.
- [16] R. L. Dykstra. An algorithm for restricted least squares regression. *Journal of the American Statistical Association*, 78(384):837–842, 1983.
- [17] J. Eckstein and D. P. Bertsekas. On the Douglas–Rachford splitting method and the proximal point algorithm for maximal monotone operators. *Mathematical programming*, 55(1):293–318, 1992.
- [18] V. Elser. Learning without loss. *Fixed Point Theory and Algorithms for Sciences and Engineering*, 2021(1):12, 2021.
- [19] V. Elser and M. K. Lal. Learning with boolean threshold functions. *arXiv preprint arXiv:2602.17493*, 2026.
- [20] L. Gubin, B. T. Polyak, and E. Raik. The method of projections for finding the common point of convex sets. *USSR Computational Mathematics and Mathematical Physics*, 7(6):1–24, 1967.
- [21] A. Guidolin, A. Lerario, I. Ren, and M. Sciamiero. Morse theory of Euclidean distance functions from algebraic hypersurfaces. *arXiv preprint arXiv:2402.08639*, 2024.
- [22] K. He, X. Zhang, S. Ren, and J. Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision*, pages 1026–1034, 2015.
- [23] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [24] G. Hinton. The forward-forward algorithm: Some preliminary investigations. *arXiv preprint arXiv:2212.13345*, 2022.

-
- [25] S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456, 2015.
 - [26] K. Jordan, Y. Jin, V. Boza, Y. Jiacheng, F. Cesista, L. Newhouse, and J. Bernstein. Muon: An optimizer for hidden layers in neural networks, 2024.
 - [27] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
 - [28] D.-H. Lee, S. Zhang, A. Fischer, and Y. Bengio. Difference target propagation. In *Joint european conference on machine learning and knowledge discovery in databases*, pages 498–515. Springer, 2015.
 - [29] G. Li and T. K. Pong. Douglas–Rachford splitting for nonconvex optimization with application to nonconvex feasibility problems. *Mathematical programming*, 159(1):371–401, 2016.
 - [30] T. P. Lillicrap, D. Cownden, D. B. Tweed, and C. J. Akerman. Random synaptic feedback weights support error backpropagation for deep learning. *Nature communications*, 7(1):13276, 2016.
 - [31] S. B. Lindstrom and B. Sims. Survey: sixty years of Douglas–Rachford. *Journal of the Australian Mathematical Society*, 110(3):333–370, 2021.
 - [32] P. Neal, C. Eric, P. Borja, and E. Jonathan. Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends® in Machine learning*, 3(1):1–122, 2011.
 - [33] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in neural information processing systems*, 2019.
 - [34] R. T. Rockafellar and R. J. Wets. *Variational analysis*. Springer, 1998.
 - [35] D. E. Rumelhart, G. E. Hinton, and R. J. Williams. Learning representations by back-propagating errors. *nature*, 323(6088):533–536, 1986.
 - [36] B. Scellier and Y. Bengio. Equilibrium propagation: Bridging the gap between energy-based models and backpropagation. *Frontiers in computational neuroscience*, 11:24, 2017.
 - [37] F. P. Such, V. Madhavan, E. Conti, J. Lehman, K. O. Stanley, and J. Clune. Deep neuroevolution: Genetic algorithms are a competitive alternative for training deep neural networks for reinforcement learning. *arXiv preprint arXiv:1712.06567*, 2017.

- [38] Y. Tang, Y. Tian, and D. Ha. Evojax: Hardware-accelerated neuroevolution. In *Genetic and Evolutionary Computation Conference Companion*, pages 308–311, 2022.
- [39] A. Themelis and P. Patrinos. Douglas–Rachford splitting and ADMM for non-convex optimization: Tight convergence results. *SIAM Journal on Optimization*, 30(1):149–181, 2020.
- [40] J. Wang, F. Yu, X. Chen, and L. Zhao. ADMM for efficient deep learning with global convergence. In *SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 111–119, 2019.
- [41] S. Xie and Z. Li. Implicit bias of AdamW: ℓ_∞ norm constrained optimization. *arXiv preprint arXiv:2404.04454*, 2024.



Code of conduct and transparency statement on the use of GenAI for KU Leuven-students (academic year 2025–2026)

Generative AI (GenAI) assistance tools can be used to generate various types of content, including text, images, code, video, music, or combinations thereof. Common examples of such tools include ChatGPT, Google Gemini, Microsoft Copilot, Midjourney, Claude.ai, Perplexity.ai, and DALL·E, among others.

This code of conduct is a tool that helps students to be transparent about the use of GenAI and fits within the university's principles on academic integrity.

Important guidelines and remarks

- **Sensitive or personal data:** Some GenAI tools protect your input and sensitive or personal data better than others. There is often no transparency on what the owners of the AI applications do with the data entered. Therefore, **do not enter sensitive or personal data in free GenAI tools**. More info about what sensitive and personal data are, is described in the [three confidentiality levels of the KU Leuven data classification model](#) (non-confidential, confidential, strictly confidential). For confidential data you should preferably use **M365 Copilot Chat with your KU Leuven account**. If you use another GenAI tool, you must be **absolutely certain** it does **not store or reuse the data you enter**. **Try to avoid entering these data. Do so only if strictly necessary, and only to the extent required. For personal data, work with anonymous or pseudonymized data.** Additionally, in case of strictly confidential data this should first be discussed with the teaching staff of the course or your thesis supervisor.
- **Copyrighted materials:** For lawfully obtained copyrighted material you should preferably use **M365 Copilot Chat with your KU Leuven account**. If you use another GenAI tool, you must be **absolutely certain** it does **not store or reuse the data you enter**.
- GenAI assistance may not be used for data or topics covered by a [Non-Disclosure Agreement \(NDA\)](#). Even in the absence of an NDA, certain information may still

need to be treated as confidential—for example, due to regulatory requirements or the risk of significant harm to the university if disclosed. In case of doubt, check with your teaching staff or supervisor.

- If your master’s thesis is under **embargo**, you should first discuss with your supervisor whether the use of GenAI is permitted.
- Before using a GenAI tool, always consider whether its use is responsible, including from a **sustainability perspective** (e.g. when using GenAI as a search engine, language assistant, ...).
- **Take a scientific and critical attitude** when interacting with GenAI assistance and interpreting its output, that may not always be correct.
- As a student you are responsible for complying with Article 84 of the Regulations on Education and Examinations: your report or thesis should reflect your own knowledge, understanding and skills. Be aware that plagiarism rules also apply to (work that is the result of) the use of GenAI assistance tools.

Exam Regulations Article 84: *“Every conduct individual students display with which they (partially) inhibit or attempt to inhibit a correct judgement of their own knowledge, understanding and/or skills or those of other students, is considered an irregularity which may result in a suitable penalty. A special type of irregularity is plagiarism, i.e. copying the work (ideas, texts, structures, designs, images, plans, codes , ...) of others or prior personal work in an exact or slightly modified way without adequately acknowledging the sources. Every possession of prohibited resources during an examination (see article 65) is considered an irregularity.”*

- In order to maintain academic integrity and avoid plagiarism, **more information about being transparent on the use of GenAI assistance and about correctly citing and referencing GenAI** can be found on this website for students ([Dutch/English](#)).
- **Additional reading:** **KU Leuven guidelines on responsible use of Generative AI tools, and other information** ([Dutch/English](#))

A few final words

If you are uncertain whether or not you should declare your use of GenAI tools, we suggest that you discuss this with your instructor or supervisor. It is always safer to declare GenAI use, even when it is not strictly required. However, declaring GenAI use does not entail that its use is allowed; the right column in the table below provides more detailed instructions in this regard (code of conduct).

Moreover, advanced AI tools are evolving rapidly, and their capabilities have expanded significantly in a short period of time. As a result, we do not yet have all

the answers about their responsible use. Finally, it is important to follow-up on the most recent evolutions in AI technologies, to have an open mind but also to be a bit cautious, to communicate with instructors, teaching assistants, supervisors and peers, to be as transparent as we can, and to learn together as we move along.

Student name: Jeff Cyuzuzo Jambé

Student number: r0902205

Please indicate with "X" whether it relates to a course assignment or to the Bachelor's or Master's thesis:

☐ This form is related to a **course assignment**.

Course name: **Course number:**

This form is related to my ☐ **Bachelor's** or ☒ **Master's thesis**.

Title Bachelor's or Master's thesis: $\mathcal{P}$ Torch Narrowing the Gap Between Projection and Gradient-Based Learning

Supervisor: Prof. dr. ir. Panos Patrinos

Daily supervisor: Ir. Jan Quan

Please indicate with "X":

☐ I did not use any GenAI assistance tool.

☒ I did use GenAI Assistance. In this case **specify which ones** (e.g. ChatGPT, M365 Copilot, ...): Gemini, Claude code, antigravity, chatgpt, vscode-copilot

GenAI assistance used as/for:	Name of the GenAI tool(s) used If helpful, also describe in which way you were using GenAI related to what is specified as code of conduct.	Code of conduct: For each of the categories below, always take into account the important guidelines and remarks mentioned above (e.g. copyrighted data, sensitive or personal data,...).
As a language assistant for reviewing or improving texts I wrote myself	claude, chatgpt, gemini	This use is similar to using spelling and grammar check tools. In general, you do not have to refer to such kind of GenAI use in the text. However, be careful: When using GenAI tools on texts you did not write yourself to improve the text, you have to refer to the original source or author, otherwise you are committing plagiarism and thus an irregularity.
As a paraphrasing tool	claude, chatgpt, gemini	This use is allowed except when it is prohibited by the teacher or the program of study. You may paraphrase your own text or texts by an author other than yourself and take inspiration from what a GenAI tool or another tool suggests (unless it is not allowed). In general, you do not have to refer to such kind of GenAI use or other paraphrasing tools in the text. However, be careful: If it entails text by an author other than yourself, you are not allowed to include that paraphrased text without reference to the original source or author. Without such reference, you would be committing plagiarism and thus an irregularity.

For translation aid to improve texts I wrote myself or to better understand text from others		This use is allowed except when it is prohibited by the teacher or the program of study. It is similar to using translation tools (Google translate, DeepL, ...). In general, you do not have to refer to such kind of GenAI use in the text. However, be careful: • You are not allowed to include that translated text without reference to the original source or author. Without such reference, you would be committing plagiarism and thus an irregularity. • Always check the translated text for correctness and meaning.
As a search engine to get information on a topic or to search for existing research on the topic	claude, chatgpt, gemini	This use is similar to e.g. a Google search or checking Wikipedia. If you write your own text based on this information, you do not have to refer to the use of GenAI in the text. You only have to refer to the existing research and references you have checked and used (without such references you would be committing plagiarism and thus committing an irregularity). However, be careful: • Be aware that the output of the GenAI tool cannot be guaranteed as a 100% reliable source of information. The output may not be entirely correct and/or be limited due to the databases it uses. Moreover, knowledge evolves and may change over time; therefore, the database of the GenAI tool may not be up to date. Therefore, verify the information and do not just copy-paste it as you should understand and critically process everything you are writing.

For literature search	claude, chatgpt, gemini	This use is comparable to e.g. a Google Scholar search. You do not have to refer to such kind of GenAI use; you only have to refer to the literature references you have checked and used (without such references you would be committing plagiarism and thus committing an irregularity). However, be careful: • Be aware that the search output is restricted to the database the GenAI tool is built on. After this initial search, look for scientific sources and conduct your own analysis of the source documents. Interpret, analyse and process the information you obtained; verify it and do not just copy-paste it as you should understand and critically process everything you are writing. • Be aware that some GenAI tools may output no or wrong references. As a student you are responsible for further checking and verifying the absence or correctness of references; do not just copy-paste it.
For generating programming code	claude code, anti-gravity, vscode-copilot; Agentic coding tools have been used very extensively in every part of the development process	Use of GenAI for coding is allowed except when it is prohibited by the teacher or the program of study. If used for coding, correctly mention the use of GenAI assistance in accordance with the instructions on the page on being transparent about the use of GenAI.

For new ideas generating (research)	claude, chatgpt, gemini ?	This use of GenAI is allowed except when it is prohibited by the teacher or the program of study. Correctly mention the use of GenAI assistance in accordance with the instructions on the page on being transparent about the use of GenAI. Be careful: • Further verify in this case whether the idea is novel or not. It is likely that it is related to existing work. If so, that existing work should be correctly referenced in the text (without such reference you would be committing plagiarism and thus committing an irregularity).
For generating synthetic data	/	Use of GenAI for generating synthetic data is allowed, provided that it is methodologically and ethically justifiable, except when it is prohibited by the teacher or the program of study. Always correctly mention the use of GenAI assistance in accordance with the instructions on the page on being transparent about the use of GenAI. Be careful: • Always carefully evaluate the generated synthetic data for quality and possible bias since the output is highly dependent on the quality of the data on which the models are trained.

For generating blocks of text (other than the allowed use without referencing mentioned above)		According to Article 84 of the Regulations on Education and Examinations your text should allow to correctly and properly assess your own knowledge, understanding and skills. Therefore, inserting blocks of text without quotes and a reference to GenAI assistance in your work is not allowed. Be careful: • If it is really needed to insert a block of text from a GenAI tool, for instance because of the nature of your assignment, mention it as a citation by using quotes and correctly mention the use of GenAI assistance in accordance with the instructions page on being transparent about the use of GenAI. • However, in general, such GenAI use should be kept to an absolute minimum; you should always check the original sources.
For generating visuals, video or audio	claude, chatgpt, gemini	Use of GenAI for generating visuals, audio or video is allowed except when it is prohibited by the teacher or the program of study. Be careful: • If used, refer to GenAI for visuals according to the usual referencing style, following the instructions on the page on referencing GenAI. • If you work with existing visuals, audio or video, that existing work should be correctly referenced in the text (without such reference you would be committing plagiarism and thus an irregularity). • Explain the usage in the methods section (if there is one) and optionally attach (or link to) the prompts with the full output (or history).

Other use (specify here; this may also include a combination of types of use mentioned above):	Co-development of mathematical proofs: Most proofs were written, completed, fixed, or verified by me using generative AI. Additionally, LLMs were used to automatically identify bugs and find working configurations, while Claude Code was used extensively to write benchmarks and conversions from math to code	To make sure other use of GenAI is allowed within the course or thesis, and if so, the conditions that may apply, contact the teaching staff of the course or the supervisor of the thesis beforehand and explain the intended GenAI use. Also inform the programme director. Motivate how you would comply with Article 84 of the exam regulations. Explain the use and the added value of the AI tool you consider to use and how it is in accordance with the assignment or thesis and the KU Leuven guidelines on responsible use of Generative AI tools. Depending on the kind of GenAI use, it may be needed to properly reference it in the text, in accordance with the instructions page on being transparent about the use of GenAI.
---	---	--